

IntelliClean AI

Complete Project Documentation

AI-powered CSV/Excel Cleaning and Live Database Query Studio

1. Project Overview and Problem Statement

IntelliClean AI is a local-first data cleaning platform that combines deterministic rules with local LLM assistance. It is designed for messy enterprise data from CSV, Excel, and live SQL databases.

The core problem is that raw business data is inconsistent, incomplete, duplicated, multilingual, and often contains OCR or formatting noise. Traditional scripts can clean syntax, but they usually miss context, so they may delete useful data or merge the wrong records.

IntelliClean solves this by profiling the dataset, understanding the semantic meaning of columns, and then applying the right repair strategy: imputation, transliteration, deduplication, outlier handling, and Golden Record generation.

2. Problems Solved by IntelliClean AI

Issue	Example	IntelliClean AI action
Exact duplicates	Same customer appears twice with identical data.	Keep one row and record the merge.
Partial duplicates	One row has a missing phone, another has it filled.	Backfill missing fields and merge.
Fuzzy duplicates	Raj Sharam vs Raj Sharmaa.	Use RapidFuzz plus supporting fields.
Missing values	Phone or salary is NULL.	Fill with rule, mean, median, mode, or leave NULL in business records.
Multilingual records	Raj Sharma and राज शर्मा.	Transliterate names and match across languages.
OCR errors	MG R0ad instead of MG Road.	Normalize OCR noise with context-aware matching.
Formatting differences	+91-9876543210 vs 9876543210.	Standardize casing, spacing, phones, and addresses.
Golden Record merge	Three partial customer rows describe one person.	Choose the best values and create one clean record.
Shared contact info	Family members share one phone number.	Compare stronger identifiers before merging.
Same name, different people	Two Vikas Singh records in different cities.	Do not merge on name alone.
Changed contact info	Same person with a new phone or email.	Merge using stable IDs and keep old values in audit history.
Low-quality records	A row contains only NULL values.	Flag for review or remove by business rule.

The goal is to improve analytics quality while preserving business identity, auditability, and downstream trust.

- 1. Exact duplicates:** identical rows like two Raj Sharma records are collapsed into one canonical record.
- 2. Partial duplicates:** records with missing fields are merged and the missing values are backfilled.
- 3. Fuzzy duplicates:** typos such as Raj Sharam vs Raj Sharma are matched with RapidFuzz and supporting fields.
- 4. Semantic duplicates:** meaning-equivalent values such as IBM and International Business Machines are linked.

- 5. Missing values:** blank fields are filled with smart imputation, or with mean, median, or mode when no rule fits.
- 6. Outliers:** extreme values such as age = 9999 are clipped with IQR-based Winsorization instead of dropping the row.
- 7. Multilingual records:** names are transliterated or translated only when needed, so entities stay intact.
- 8. OCR errors:** scan noise such as MG R0ad or De1hi is normalized before matching.
- 9. Formatting differences:** variants like +91-9876543210, 91 9876543210, and 9876543210 are standardized.
- 10. Golden Record merges:** duplicate clusters are merged into the most complete row while keeping the audit trail.
- 11. Shared contact details:** records that share a phone number are compared with stronger identifiers before merging.
- 12. Low-quality records:** rows with almost no usable data are flagged for review or removal by business rules.

3. Executive Summary

IntelliClean AI is a local-first data quality platform that combines deterministic data cleaning with local LLM assistance. It is built to handle messy CSV, Excel, and SQL database data while preserving business meaning, auditability, and user trust.

The system is split into two primary modes: a file-based cleaning workflow for spreadsheets and a live database studio for connected SQL sources. In both modes, the user can profile data, understand schema meaning, resolve duplicates, handle multilingual values, impute missing data, clip outliers, and export a cleaned result with a report.

The supplied documentation shows a platform designed for end-to-end data remediation rather than a single transform script. It has a clear UI, a central pipeline orchestrator, a modular backend, and explainability built into the final output.

- Offline execution is a major design goal: local inference is intended to run through an LM Studio-compatible server.
- The cleaning pipeline favors preservation over destruction, especially when repairing duplicates and outliers.
- Every major action is intended to produce metadata for auditing, validation, and report generation.

4. Features and Functionalities

IntelliClean AI is built to clean enterprise data locally, explain its decisions, and preserve business meaning.

- 100% local AI integration through LM Studio for offline, privacy-safe inference.
- Universal ingestion for CSV, Excel, and live SQL databases with memory-protected loading.
- Dynamic multi-agent reasoning for schema profiling, rule planning, validation, and SQL generation.
- Multilingual translation and transliteration with entity preservation.
- Advanced fuzzy deduplication using Sorted Neighbourhood Indexing and RapidFuzz.
- Entity resolution that creates a single Golden Record instead of deleting useful data.
- IQR-based outlier handling with Winsorization to keep rows intact.
- Real-time Streamlit UI that streams logs and AI reasoning to the user.

7. Current Project Structure

The codebase is split into a thin UI layer, reusable agents, the backend pipeline, and the cleaning engine.

```
IntelliClean/
├── app.py                                # Main Streamlit Frontend & CSV/Excel Orchestrator
├── pages/
│   └── Live_Database.py                 # Live DB Connection & NL Query Console UI
├── config.py                            # Global hyperparameters and API endpoints
├── requirements.txt                      # Dependency list
├── agents/                              # The AI Cognitive Engine
│   ├── llm_client.py                   # Unified LM Studio HTTP interface
│   ├── schema_agent.py                 # Semantic column profiling & tier identification
│   ├── planner_agent.py                # Imputation rule generation
│   ├── nl_query_agent.py               # Natural language to SQL query generation
│   ├── validation_agent.py             # Post-clean logic auditing
│   └── explanation_agent.py            # Human-readable markdown generation
├── backend/                             # Infrastructure & Orchestration
│   ├── pipeline.py                     # Central 12-phase orchestrator
│   ├── loader.py                       # PyArrow ingestion & missing value coercion
│   ├── db_connector.py                 # SQLAlchemy connector for Live DBs (MySQL, Postgres, etc)
│   ├── schema_detector.py              # Fast heuristic datatype & schema detection
│   ├── domain_profiler.py              # Industry context detection
│   ├── profiler.py                     # Baseline statistical calculation
│   ├── validator.py                    # Hardcoded math/constraint checks
│   ├── exporter.py                     # File I/O for reports and CSVs
│   ├── config_updater.py               # Modifies config settings programmatically
│   └── state_manager.py                # Connection persistence & session state handling
├── cleaning/                            # The Core Algorithms
│   ├── quality_filter.py               # >90% sparsity & dummy data dropping
│   ├── datatype_cleaner.py             # Mixed-type normalization and boolean mapping
│   ├── multilingual.py                 # O(1) Token Caching translation & transliteration
│   ├── currency_converter.py           # Regex symbol extraction & INR mapping
│   ├── standardizer.py                 # Semantic formatting (lowercase emails, etc)
│   ├── string_cleaner.py               # Zero-width char & whitespace removal
│   ├── deduplication.py                # Sorted Neighbourhood fuzzy matching (16 rules)
│   ├── entity_resolution.py            # Cluster merging and primary row backfilling
│   ├── missing_values.py               # AI rule evaluation + Skewness statistical fills (fill_none)
│   └── outliers.py                     # IQR calculation and Winsorization (Clipping)
├── components/                          # Streamlit Custom UI Components
│   ├── settings_modal.py               # Pop-up modal for global configs (LLM, levels)
│   └── ui_components.py                # Premium UI cards, metrics, and log rendering
└── All Scripts Working Guide/           # Comprehensive script-by-script markdown guides
```

8. Technology Stack and Libraries

Technology Stack & Libraries used

The IntelliClean AI platform is built using a collection of modern Python libraries and technologies that enable intelligent data cleaning, deduplication, multilingual processing, live database connectivity, and an interactive web interface.

Application Framework (Frontend)

- **Streamlit** – Provides the interactive web application, including dashboards, file upload, dataset visualization, real-time pipeline logs, and report downloads.

Artificial Intelligence & Validation

- **Requests** – Connects to the local LM Studio REST API for AI-assisted schema analysis, SQL generation, smart imputation planning, and explanations.
- **Pydantic** – Validates and structures AI-generated JSON responses, ensuring reliable communication between the application and the local LLM.

Data Processing & Engineering

- **Pandas** – Core library for DataFrame manipulation, profiling, cleaning, filtering, transformation, and report generation.
- **NumPy** – Performs efficient numerical computations, statistical calculations, and missing-value handling.
- **PyArrow** – High-performance backend for fast CSV/Excel processing, optimized memory usage, and efficient data loading.
- **OpenPyXL** – Reads and writes Microsoft Excel (.xlsx) files.

Deduplication & Entity Resolution

- **RecordLinkage** – Performs candidate record generation (blocking/indexing) and similarity comparison for duplicate detection.
- **RapidFuzz** – Implements fast fuzzy string matching to detect typographical variations, OCR errors, and near-duplicate records.
- **Unidecode** – Normalizes multilingual Unicode text into ASCII equivalents, improving cross-language matching and transliteration.

Database Connectivity

- **SQLAlchemy** – Unified ORM and database toolkit used to connect, inspect schemas, and execute SQL queries across multiple database systems.

Database Drivers

- **PyMySQL** - MySQL and MariaDB connectivity.
- **Psycopg2-binary** - PostgreSQL connectivity.
- **PyODBC** - Microsoft SQL Server connectivity.
- **OracleDB** - Oracle Database connectivity.
- **IBM_DB_SA** - IBM DB2 SQLAlchemy adapter.

System Monitoring & Utilities

- **Psutil** - Monitors CPU usage, memory consumption, and system resources to support memory-aware processing of large datasets.
- **Tabulate** - Generates well-formatted tabular reports for console output and debugging.

9. Data Cleaning Project Workflow

Load & Profile -> AI Schema -> Strategy Plan -> Pre-Cleaning -> Translation -> Entity Resolution -> Domain Rules -> Deduplication -> Imputation -> Outliers -> Validation -> Audit Trail

10. Project Overview and Architecture

The platform is centered on a Streamlit frontend that streams pipeline progress in real time while a collection of backend modules performs profiling, cleaning, reasoning, validation, and export. The architecture is intentionally modular so that each concern can be improved independently without changing the entire application.

The pipeline stages are easy to read from the screenshots and markdown files: load data, compute a health profile, classify semantics, select a strategy, standardize content, resolve multilingual and duplicate records, impute values, clip outliers, validate the result, and export outputs. This order matters because later stages depend on the quality of earlier normalization steps.

The project also separates deterministic logic from AI-assisted logic. Heuristics are used first for speed and stability, while local LLM calls are reserved for places where semantic context is needed, such as schema interpretation, rule planning, explanations, and text-to-SQL generation.

- Frontend: Streamlit dashboard with live progress, tabs, metrics, and download actions.
- Orchestration: `pipeline.py` coordinates the ordered execution of cleaning phases.
- AI services: agents for schema, planning, validation, explanation, and SQL generation.
- Data engine: loader, profiler, standardizers, deduplication, entity resolution, imputation, outlier handling, and export.

11. Global Configuration and Runtime Settings

The global configuration layer centralizes the system settings that define behavior across the whole application. This includes missing-value markers, the local LLM endpoint, retry limits, fuzzy-matching thresholds, and outlier clipping settings. The screenshot below shows the user-facing configuration dialog that exposes those values in the app.

Treating these values as shared configuration keeps the codebase easier to maintain. The same logic can be reused across file-based cleaning and live database cleaning without hardcoding thresholds in multiple scripts.

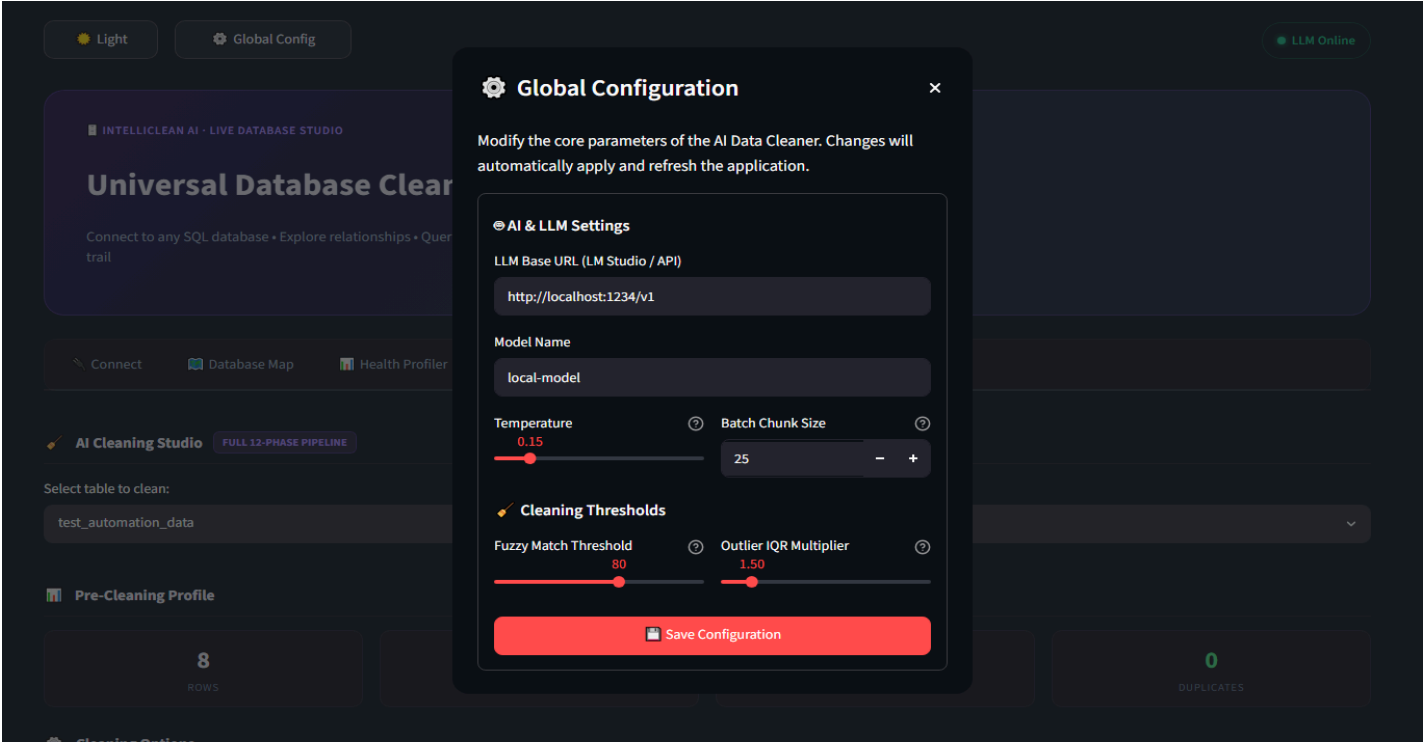


Figure 1. Global configuration modal for LLM, cleaning thresholds, and execution settings.

Setting group	Meaning
LLM / AI	Controls the local OpenAI-compatible base URL, model name, temperature, and batch chunk size.
Cleaning thresholds	Exposes fuzzy-match similarity and outlier IQR multiplier values for the cleaning engine.
Operational control	Lets the user save updated configuration values and refresh the application behavior.

12. Mode 1: CSV and Excel Based Cleaning

Mode 1 is the file-based workflow. Users upload a spreadsheet, inspect the raw data, review statistics and missingness, start the AI cleaning pipeline, and then review the cleaned dataset and audit trail before downloading the final outputs.

The screenshots in this section are intentionally ordered to match the user flow exactly. They begin with the landing page, move through ingestion and profiling, and end with the final cleaned-data and audit-trail views.

9.1 Landing Screen and Product Introduction

This is the main landing view for the spreadsheet workflow. It introduces IntelliClean AI and frames the app as an AI-powered, multilingual, universal cleaner for CSV and Excel files.

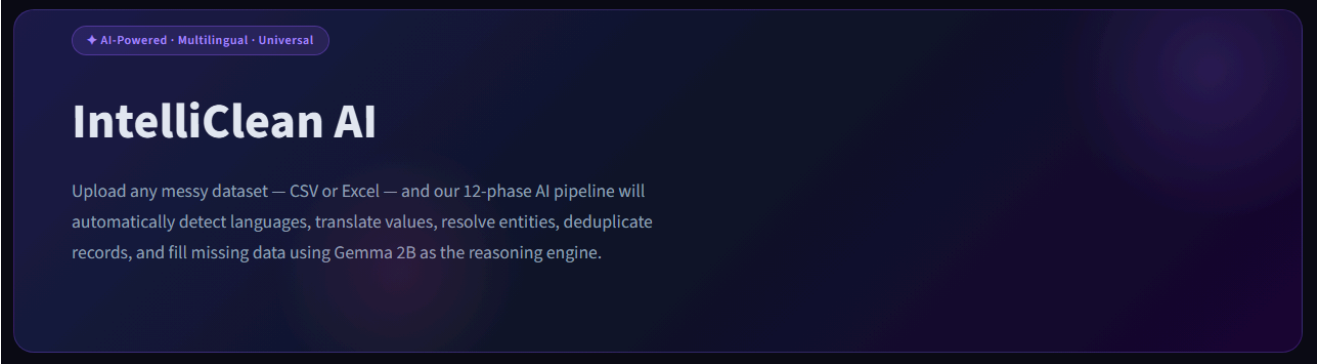


Figure. 9.1 Landing Screen and Product Introduction.

Interface note	Details
What this screen shows	The screen shows the hero banner, product name, and short value proposition before any data is loaded.
Visible UI elements	Large title panel, short marketing summary, and a clean empty-state layout that tells the user the system is ready to begin.
Why it matters	It sets the context for the full workflow and helps users understand that the application is designed for end-to-end automated cleaning.
Backend mapping	Rendered by app.py, which initializes the Streamlit interface and prepares the session state for the cleaning pipeline.
Next step	The user proceeds to upload a CSV or Excel file from the Data Ingestion section.

9.2 Data Ingestion

This screen is the entry point for file-based processing. It allows the user to choose the file-based mode and upload a dataset for cleaning.

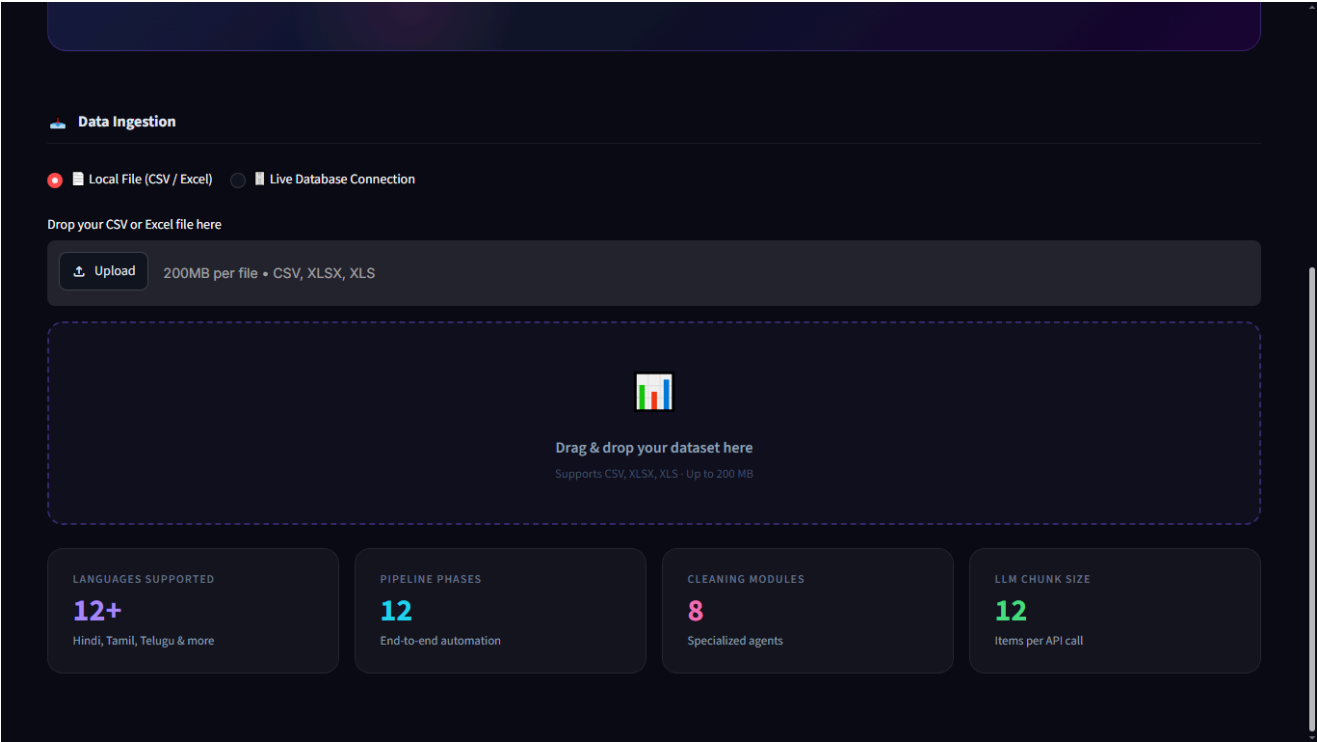


Figure. 9.2 Data Ingestion.

Interface note	Details
What this screen shows	The interface presents local file upload support, drag-and-drop upload, and summary counters showing the platform scale.
Visible UI elements	Local File / Live Database mode toggle, upload button, drag-and-drop area, supported file formats, and cards showing languages supported, pipeline phases, cleaning modules, and LLM chunk size.
Why it matters	This view confirms that the application can handle spreadsheet data directly and prepares the data for profiling and transformation.
Backend mapping	The uploaded file is handled by the frontend, then backend/loader.py reads the file and normalizes missing markers before profiling begins.
Next step	After upload, the user opens the dataset preview and statistics panel.

9.3 Dataset Preview and Statistics Overview

After ingestion, the user reaches the first data inspection workspace. This view provides the first look at the raw table and the dataset summary.

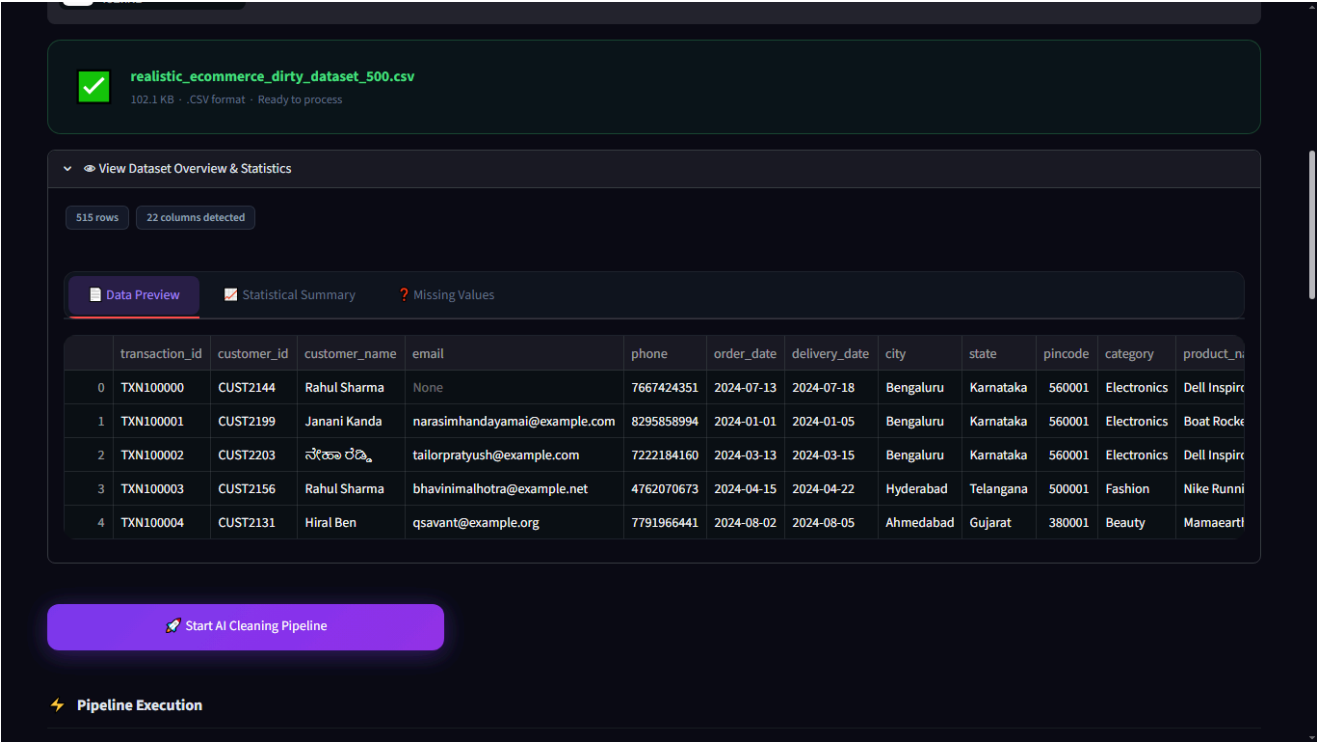


Figure. 9.3 Dataset Preview and Statistics Overview.

Interface note	Details
What this screen shows	The screen shows the uploaded file name, row and column counts, and a tabbed preview area for data preview, statistical summary, and missing values.
Visible UI elements	Dataset header, row/column count badges, tab bar, and a multi-column table preview of the uploaded file.
Why it matters	It lets the user verify that the correct file was loaded and gives an early sense of data completeness and shape before cleaning begins.
Backend mapping	app.py and backend/profiler.py provide the row/column summary, while the preview originates from the loaded DataFrame.
Next step	The user can switch tabs to inspect statistical summary or missing-value distribution.

9.4 Statistical Summary

This tab gives a statistical snapshot of the numeric fields in the dataset. It is a quick health check before AI cleaning starts.

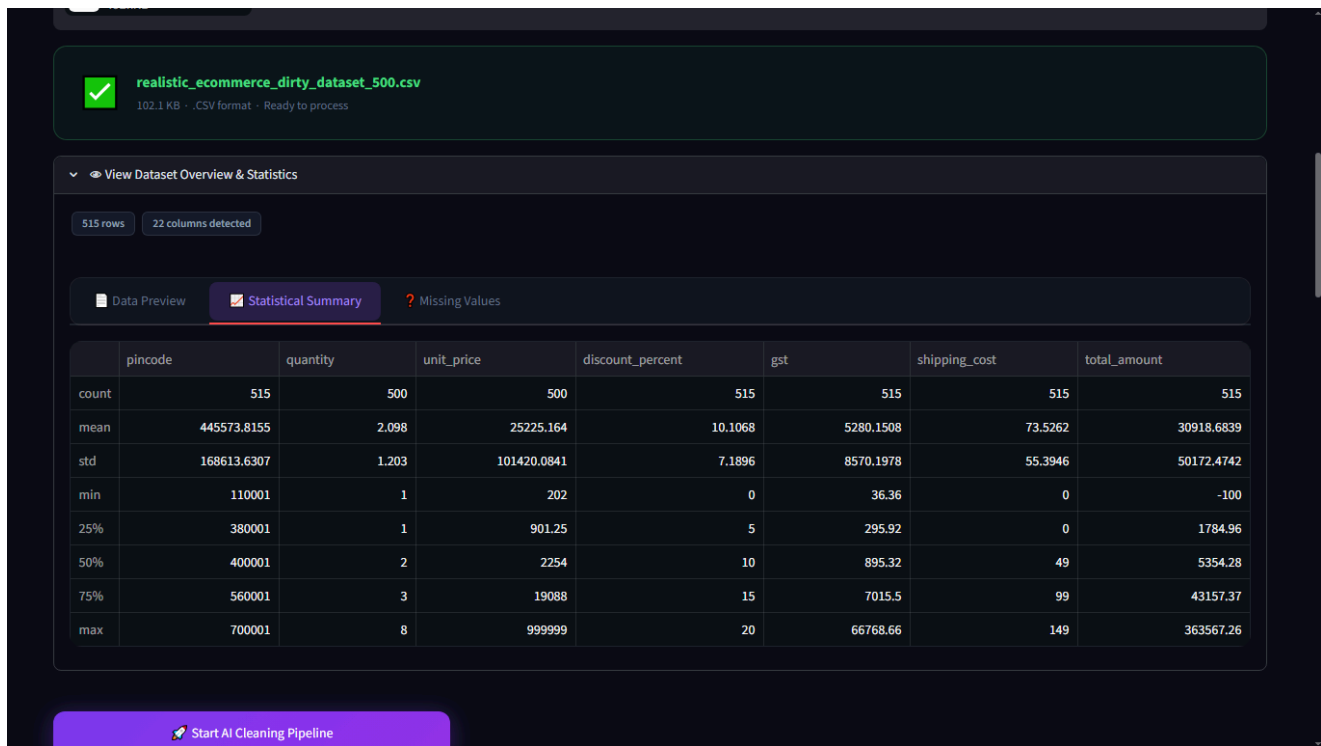


Figure. 9.4 Statistical Summary.

Interface note	Details
What this screen shows	The table shows count, mean, standard deviation, min, quartiles, and max values for columns such as pincode, quantity, unit_price, gst, shipping_cost, and total_amount.
Visible UI elements	A summary table with classic descriptive statistics and the same dataset header and tab navigation above it.
Why it matters	It helps the user spot anomalies such as extreme unit prices or unusual totals before the pipeline modifies the data.
Backend mapping	backend/profiler.py computes these statistics and stores them in the metadata for display in the UI.
Next step	The user can compare these values with the missing-value view or move directly into pipeline execution.

9.5 Missing Values Analysis

This tab highlights missingness at the column level. It shows which fields have incomplete data and how much of the dataset is affected.

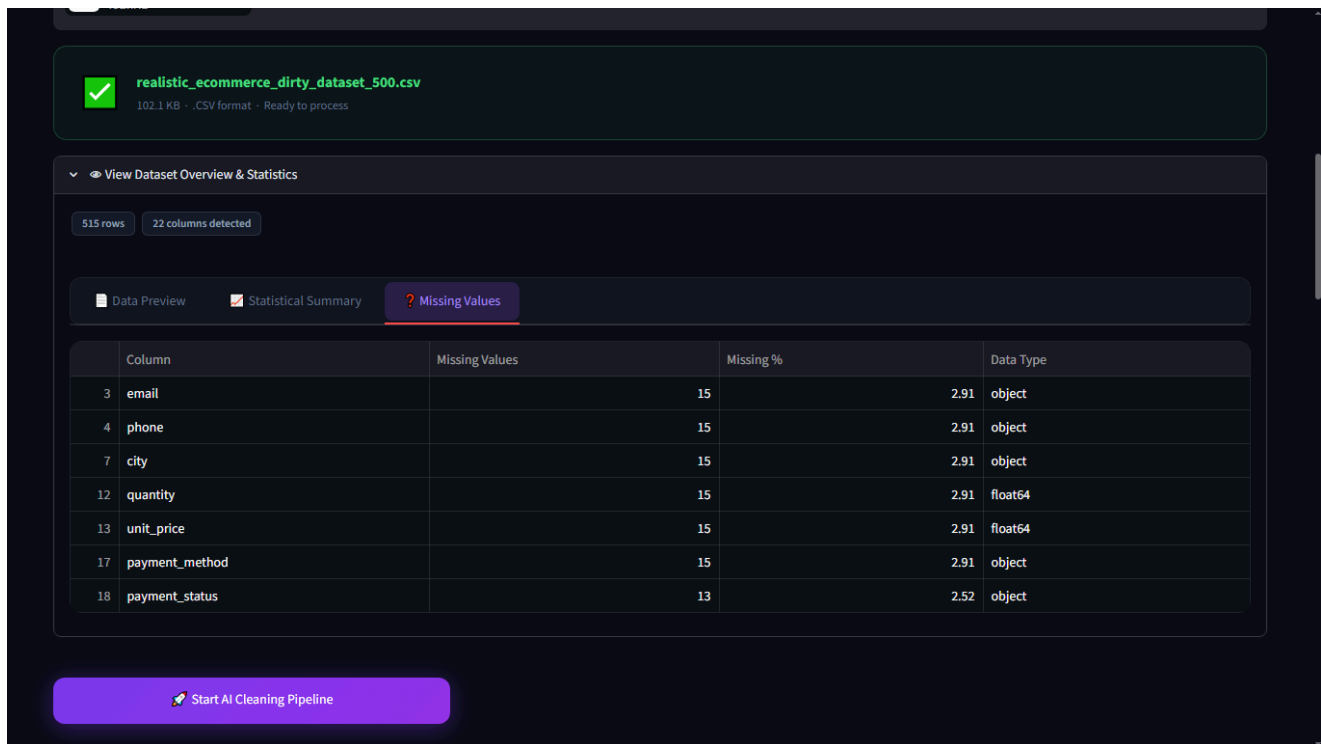


Figure. 9.5 Missing Values Analysis.

Interface note	Details
What this screen shows	The table lists column names, missing counts, missing percentages, and dtypes for fields such as email, phone, city, quantity, unit_price, payment_method, and payment_status.
Visible UI elements	Column-by-column null table, percentage metrics, and the same navigation and dataset header from the profiling area.
Why it matters	It identifies the fields that should be repaired by the imputation engine and helps explain why a dataset score is lower before cleaning.
Backend mapping	backend/profiler.py and cleaning/missing_values.py use this information to decide whether a column should be filled by rules, mean/median, or mode.
Next step	The user can start the AI cleaning pipeline to repair these gaps automatically.

9.6 Pipeline Execution Log

This is the operational view of the 12-phase pipeline. It shows progress, logs, and completion status while cleaning is underway.

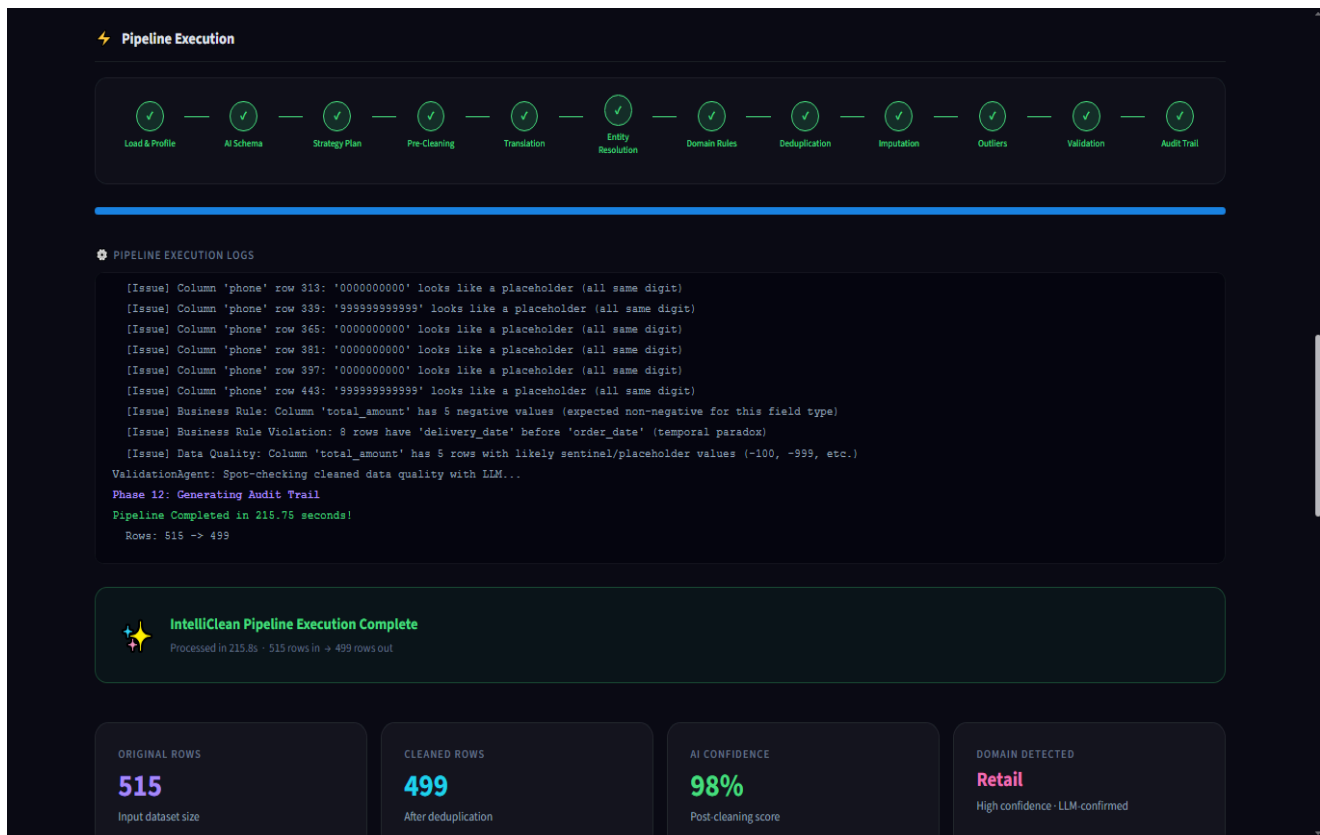


Figure. 9.6 Pipeline Execution Log.

Interface note	Details
What this screen shows	The top tracker displays Load and Profile, AI Schema, Strategy Plan, Pre-Cleaning, Translation, Entity Resolution, Domain Rules, Deduplication, Imputation, Outliers, Validation, and Audit Trail.
Visible UI elements	Green stage indicators, live log output, completion notice, processed row counts, confidence score, and detected business domain.
Why it matters	It makes the pipeline transparent instead of hidden, so users can watch the AI and deterministic logic work together in real time.
Backend mapping	backend/pipeline.py orchestrates the stages and pushes log messages into the Streamlit UI through the callback logger.
Next step	When the run finishes, the user reviews the cleaned data, explanation report, and audit trail.

9.7 Cleaned Dataset Preview - First View

This screen shows the cleaned dataset after the first major pass of the pipeline. It is the point where the transformed values become visible to the user.

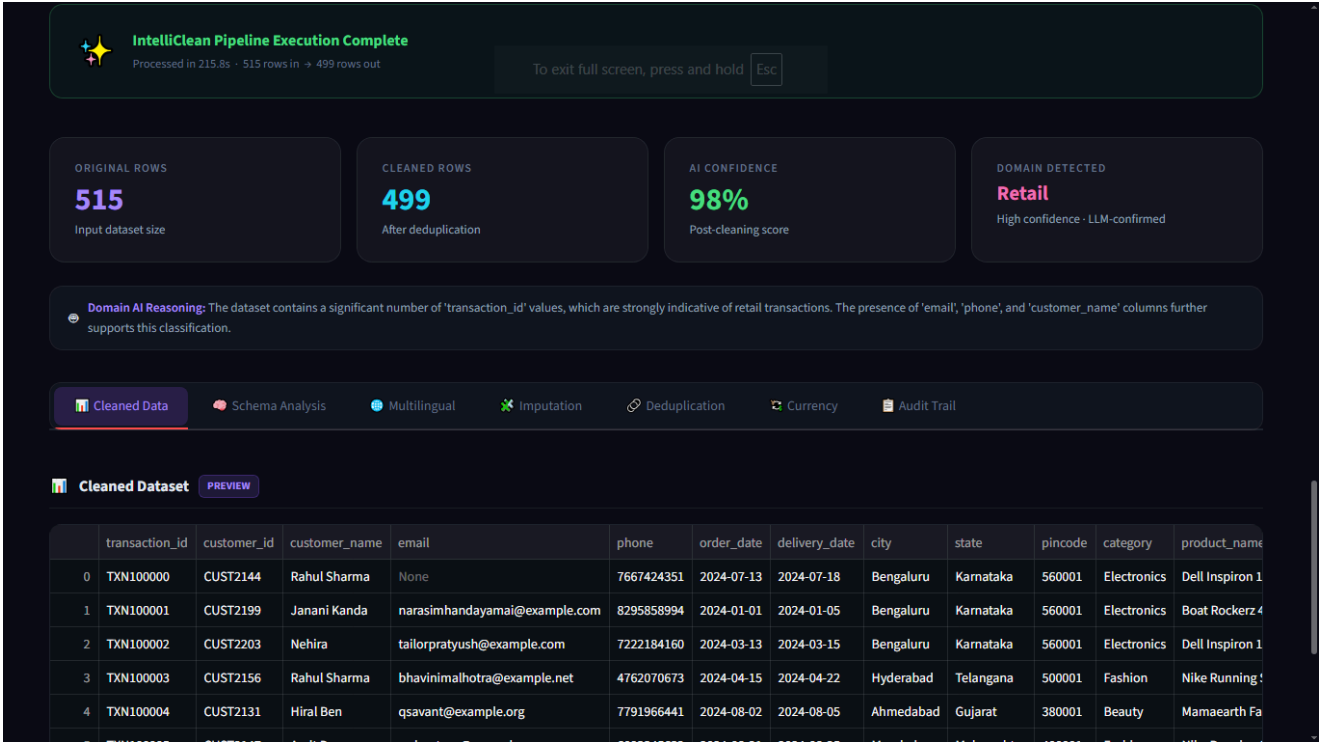


Figure. 9.7 Cleaned Dataset Preview - First View.

Interface note	Details
What this screen shows	The interface displays cleaned rows, updated columns, and the post-cleaning dataset table with the tabbed analytics panel above it.
Visible UI elements	Cleaned data preview, updated summary cards, tabs for schema analysis, multilingual processing, imputation, deduplication, currency, and audit trail.
Why it matters	It confirms that the cleaning pipeline has actually changed the data rather than only logging messages.
Backend mapping	The final DataFrame is returned by backend/pipeline.py and rendered in the Streamlit UI through app.py.
Next step	The user can compare the first cleaned preview with the second cleaned data screen for a more complete view.

9.8 Cleaned Dataset Preview - Second View

This second cleaned-data view gives a fuller table preview and reinforces that the pipeline output is ready for downstream use.

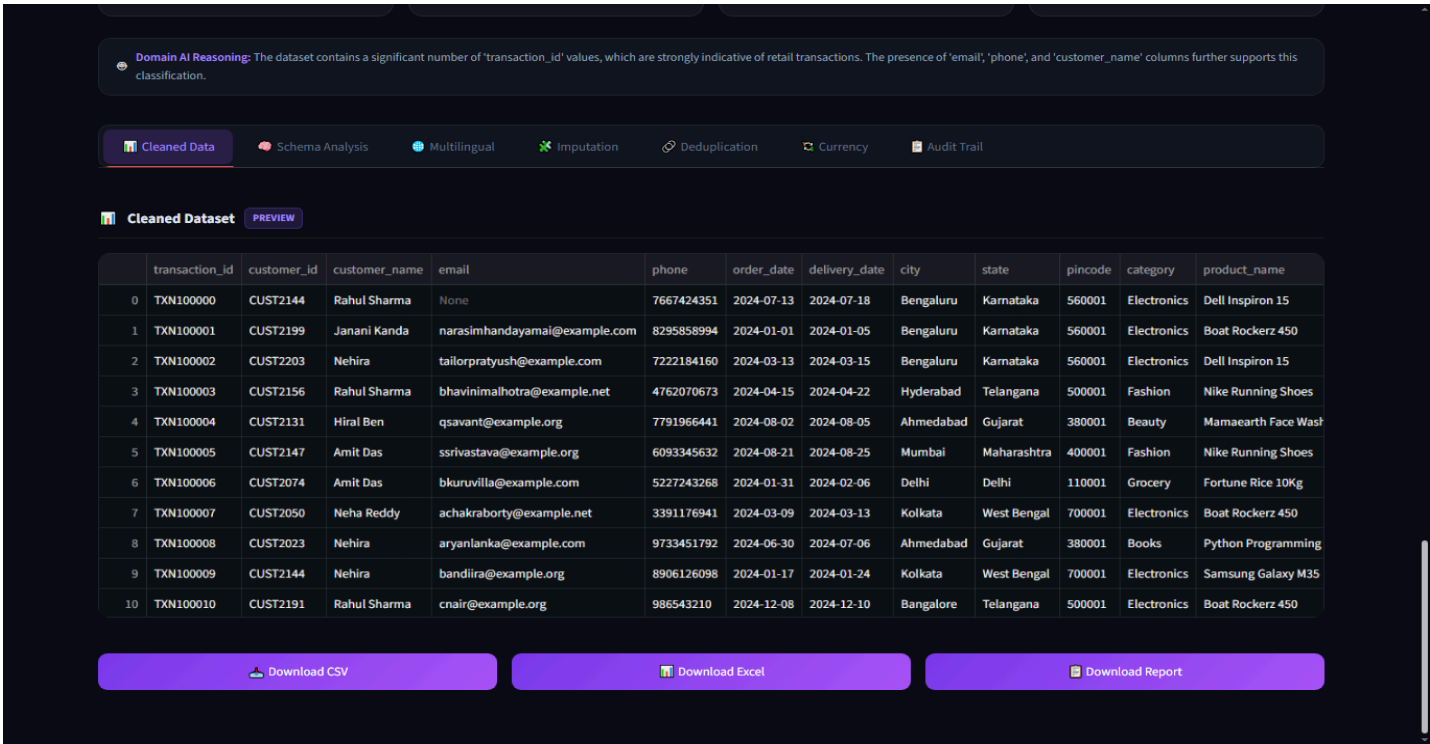


Figure. 9.8 Cleaned Dataset Preview - Second View.

Interface note	Details
What this screen shows	The table shows the cleaned records in a more detailed scrollable view with the same analysis tabs available for deeper inspection.
Visible UI elements	Large cleaned DataFrame preview and summary cards that reflect the dataset after transformation.
Why it matters	A second preview is useful because it gives the user confidence that the outputs are stable and readable before exporting.
Backend mapping	The cleaned result comes from the same post-processing DataFrame that powers downloads and reports.
Next step	The user can inspect semantic classification and special cleaning tabs such as schema, multilingual, and imputation.

9.9 Schema Analysis

This is the semantic classification view. It shows how IntelliClean interprets each column rather than just detecting its datatype.

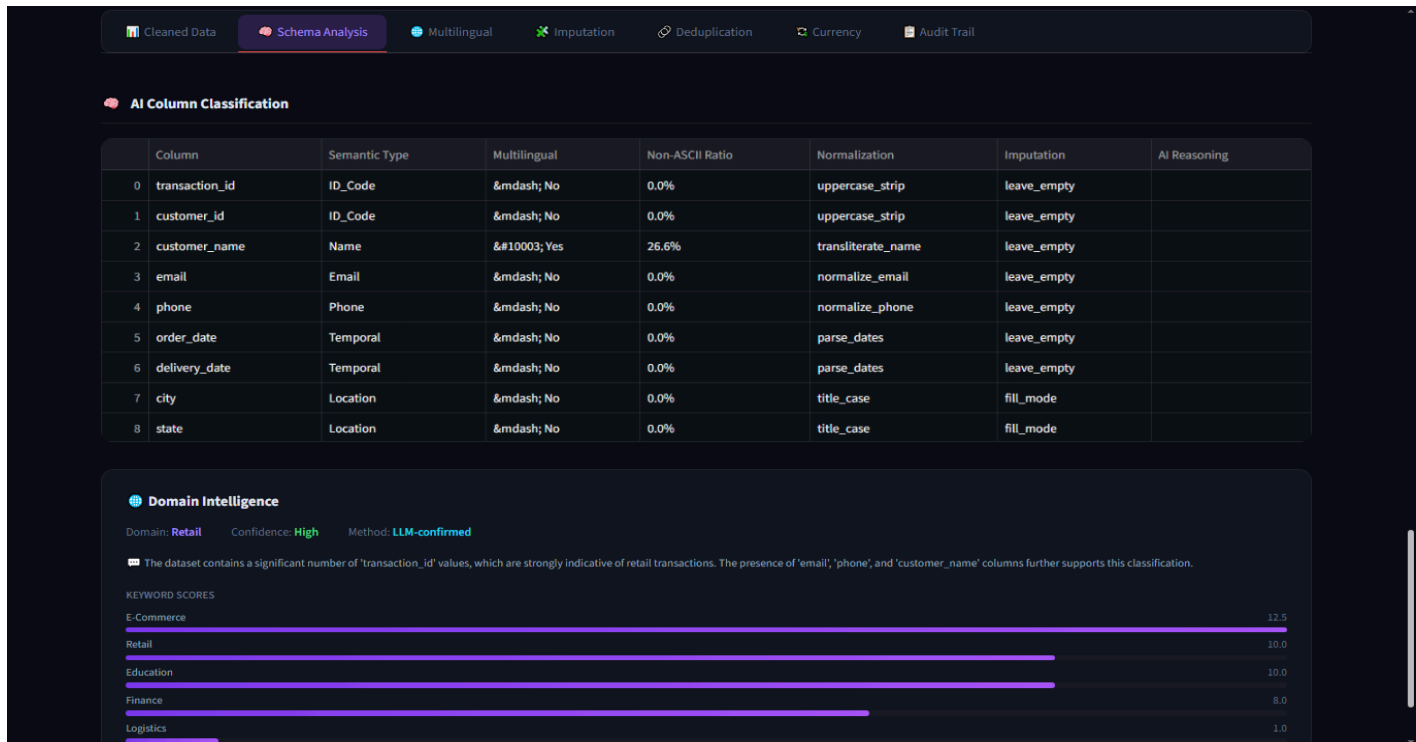


Figure. 9.9 Schema Analysis.

Interface note	Details
What this screen shows	The table maps columns to semantic types such as ID_Code, Name, Email, Phone, Temporal, and Location, and it records multilingual status, non-ASCII ratio, normalization rule, and imputation hint.
Visible UI elements	AI Column Classification table, domain intelligence panel, and confidence-based domain explanation for the dataset.
Why it matters	This is the foundation for all later reasoning because schema meaning determines how the system should normalize, translate, deduplicate, or impute a field.
Backend mapping	backend/schema_detector.py performs fast heuristics, and agents/schema_agent.py confirms the semantic type through local AI reasoning.
Next step	The user can move into multilingual processing, imputation, or deduplication based on these schema labels.

9.10 Multilingual Detection and Translation

This screen shows how the pipeline handles multilingual entity values. It makes the translation or transliteration step visible instead of hiding it inside the backend.

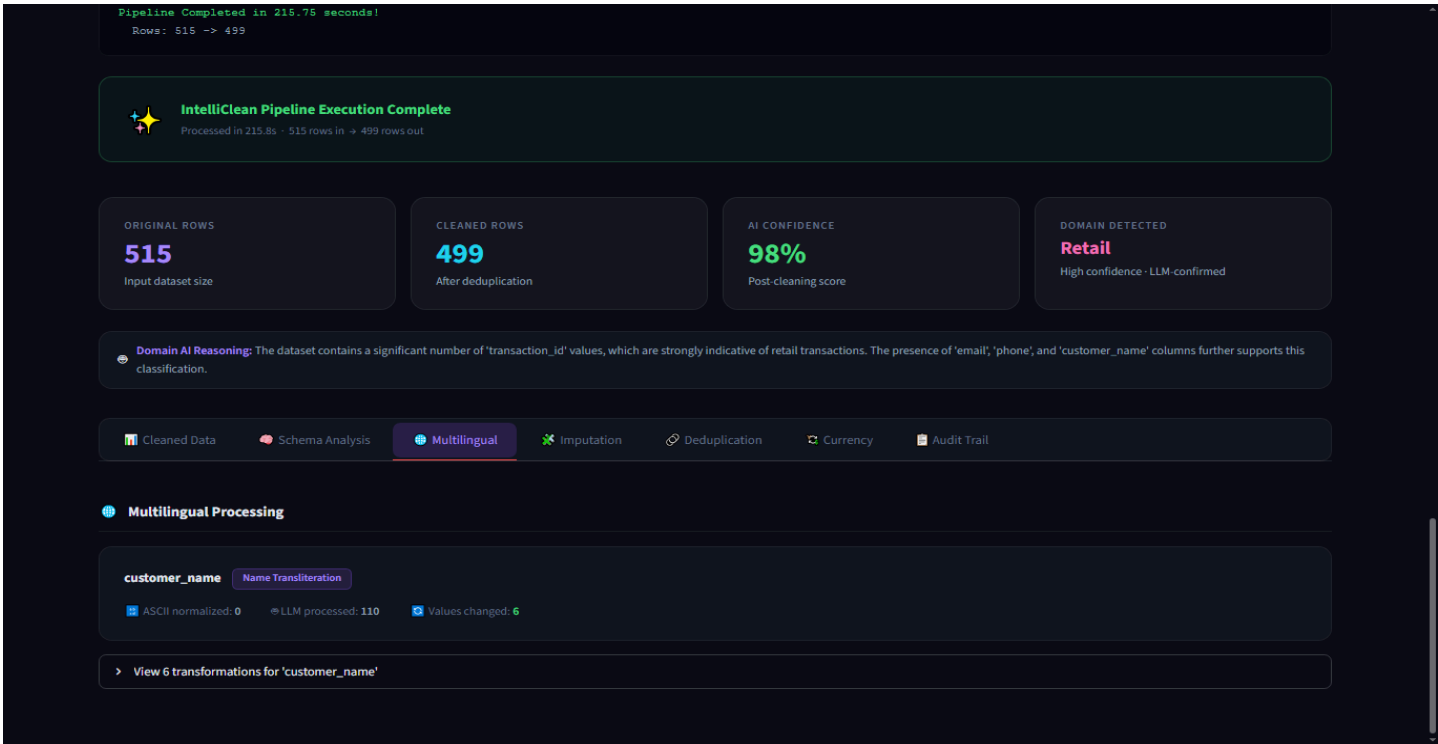


Figure. 9.10 Multilingual Detection and Translation.

Interface note	Details
What this screen shows	The panel shows the post-cleaning outcome with cleaned row counts and a multilingual processing section for the customer_name field, including the number of processed and changed values.
Visible UI elements	Multilingual tab, per-column transformation summary, and a collapsible record of the detected changes.
Why it matters	It demonstrates that names and entities are preserved while non-ASCII or cross-language values are normalized for matching.
Backend mapping	cleaning/multilingual.py and the audit metadata together record the transformations made by transliteration or translation logic.
Next step	The user can open the transformation details or continue to imputation and duplicate consolidation.

9.11 Smart Imputation

This view explains how missing values were filled. It combines AI-generated contextual rules with statistical fallback methods when needed.

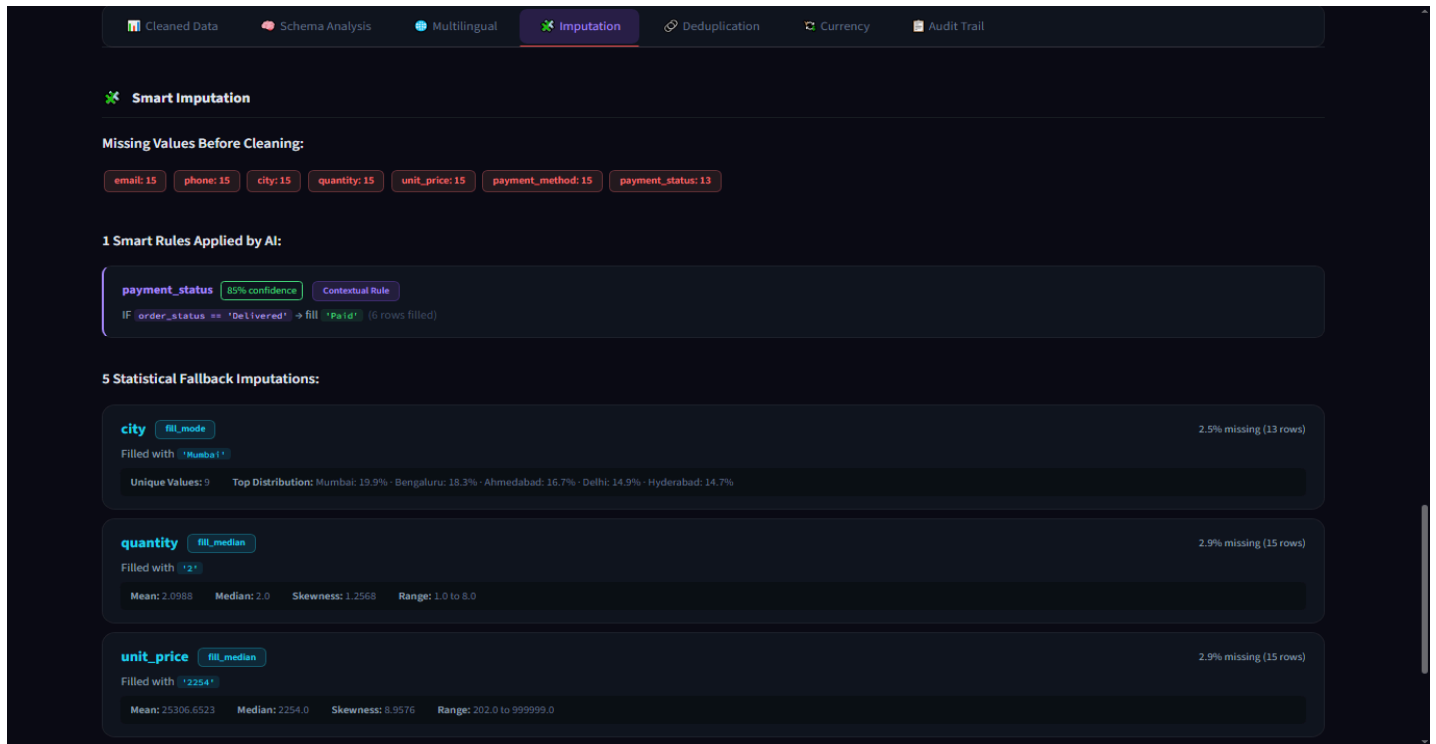


Figure. 9.11 Smart Imputation.

Interface note	Details
What this screen shows	The screen lists missing values before cleaning, shows an applied AI rule with confidence, and then displays column-level fallback imputations such as mode and median.
Visible UI elements	Missing-value badges, a smart rule card, and detailed fallback cards that report fill strategy, summary statistics, and the number of rows affected.
Why it matters	It makes the imputation decision explainable and helps the user trust that values were filled using domain-aware logic rather than arbitrary defaults.
Backend mapping	agents/planner_agent.py proposes rules and cleaning/missing_values.py applies them before falling back to mean, median, or mode.
Next step	The user can inspect deduplication and outlier handling after missing values have been repaired.

9.12 Deduplication and Entity Consolidation

This screen shows duplicate resolution and golden-record creation. It turns several similar records into a single, richer entity record.

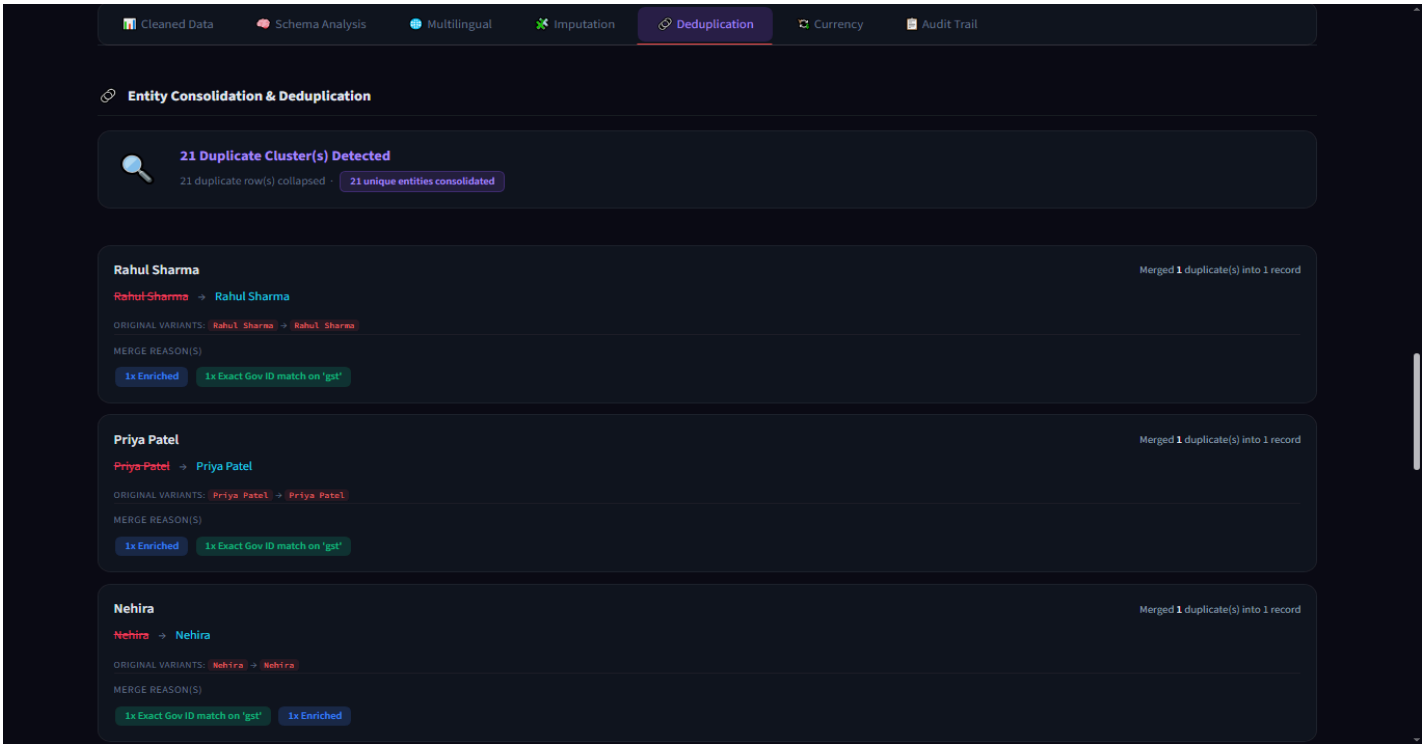


Figure. 9.12 Deduplication and Entity Consolidation.

Interface note	Details
What this screen shows	The panel reports the number of duplicate clusters detected and shows individual merge cards for people such as Rahul Sharma, Priya Patel, and Nehira, including original variants and merge reasons.
Visible UI elements	Cluster count, merged record cards, original variant labels, and rationale tags such as enrichment or exact Gov ID match.
Why it matters	It proves that IntelliClean preserves data by consolidating duplicates instead of blindly deleting rows.
Backend mapping	cleaning/deduplication.py detects clusters and cleaning/entity_resolution.py backfills the canonical record with missing values from related rows.
Next step	The user can review currency conversion and the audit trail for a full change history.

9.13 Currency Normalization

This screen captures the financial standardization stage. It is used when a dataset contains mixed currency formats or currency symbols that must be normalized.

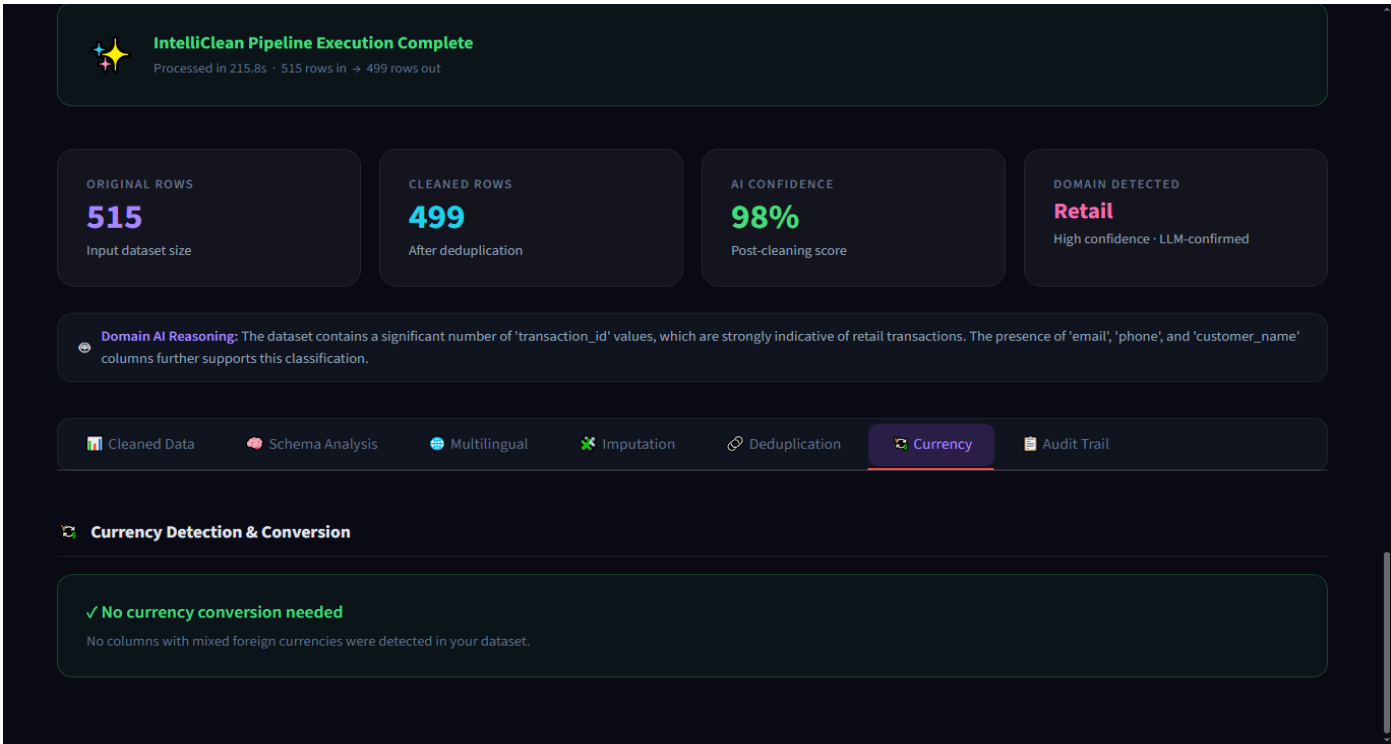


Figure. 9.13 Currency Normalization.

Interface note	Details
What this screen shows	The dashboard presents cleaning metrics and shows how currency-related values are standardized as part of the broader transformation pipeline.
Visible UI elements	Summary metrics, currency-oriented controls or results, and the same analysis tabs used in the rest of the workflow.
Why it matters	Mixed-currency columns often block numeric analysis, so normalization is needed before outlier handling or reporting.
Backend mapping	cleaning/currency_converter.py strips symbols, coerces numeric values, and applies the configured conversion mapping.
Next step	After normalization, the user can inspect the alternate currency view and final audit trail.

9.14 Currency Normalization - Alternate View

This is a second currency-related capture from the workflow archive. It reinforces that the app can present financial normalization in more than one results view.

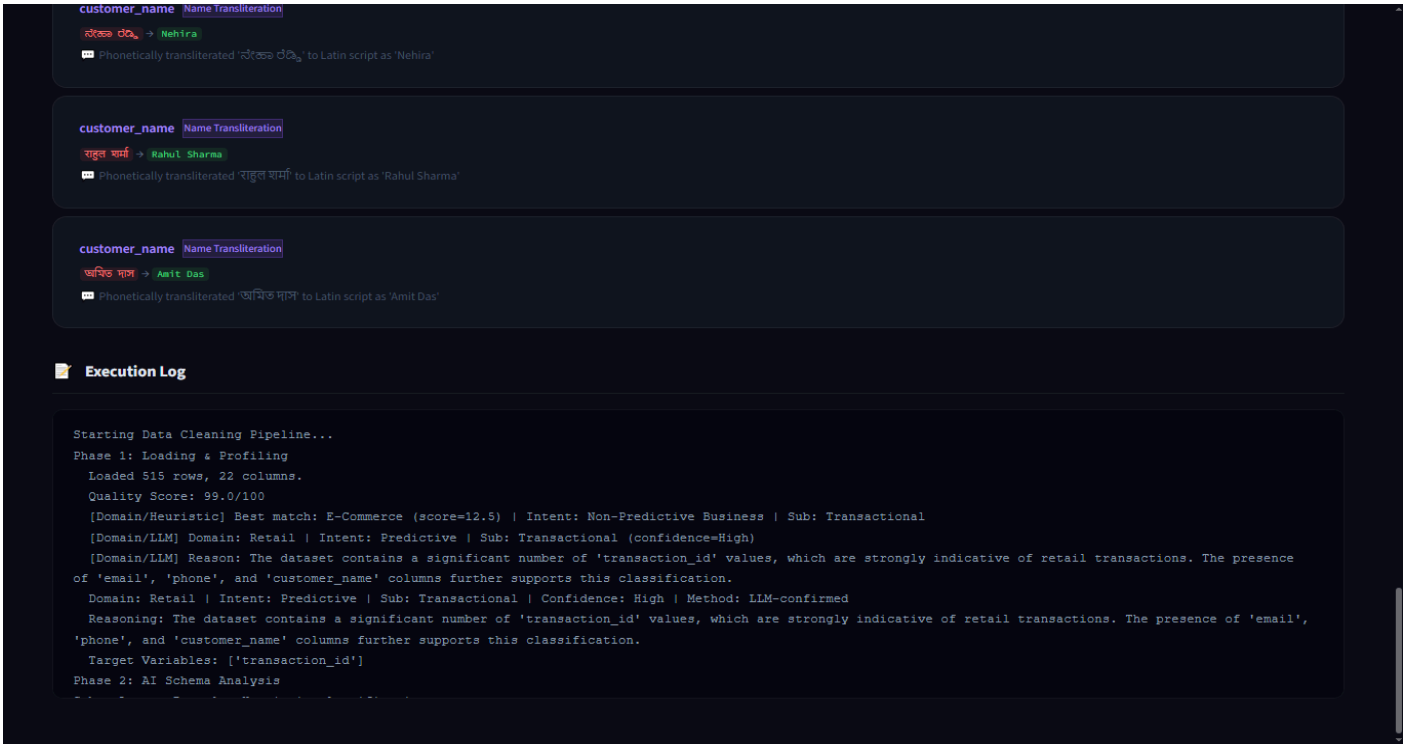


Figure. 9.14 Currency Normalization - Alternate View.

Interface note	Details
What this screen shows	The screen again centers the currency stage inside the pipeline navigation, showing how the app keeps specialized transformation steps visible to the user.
Visible UI elements	Pipeline tabs, result cards, and a currency-focused status panel.
Why it matters	Repeated capture of the same stage is useful because it shows the same transformation from a slightly different point in the workflow.
Backend mapping	The same currency standardization logic from cleaning/currency_converter.py is referenced in the pipeline metadata.
Next step	The user can finish by opening the audit trail to review every transformation made by the system.

9.15 AI Audit Trail

This screen records the explainability layer. It shows exactly what changed, why it changed, and how the AI described each transformation.

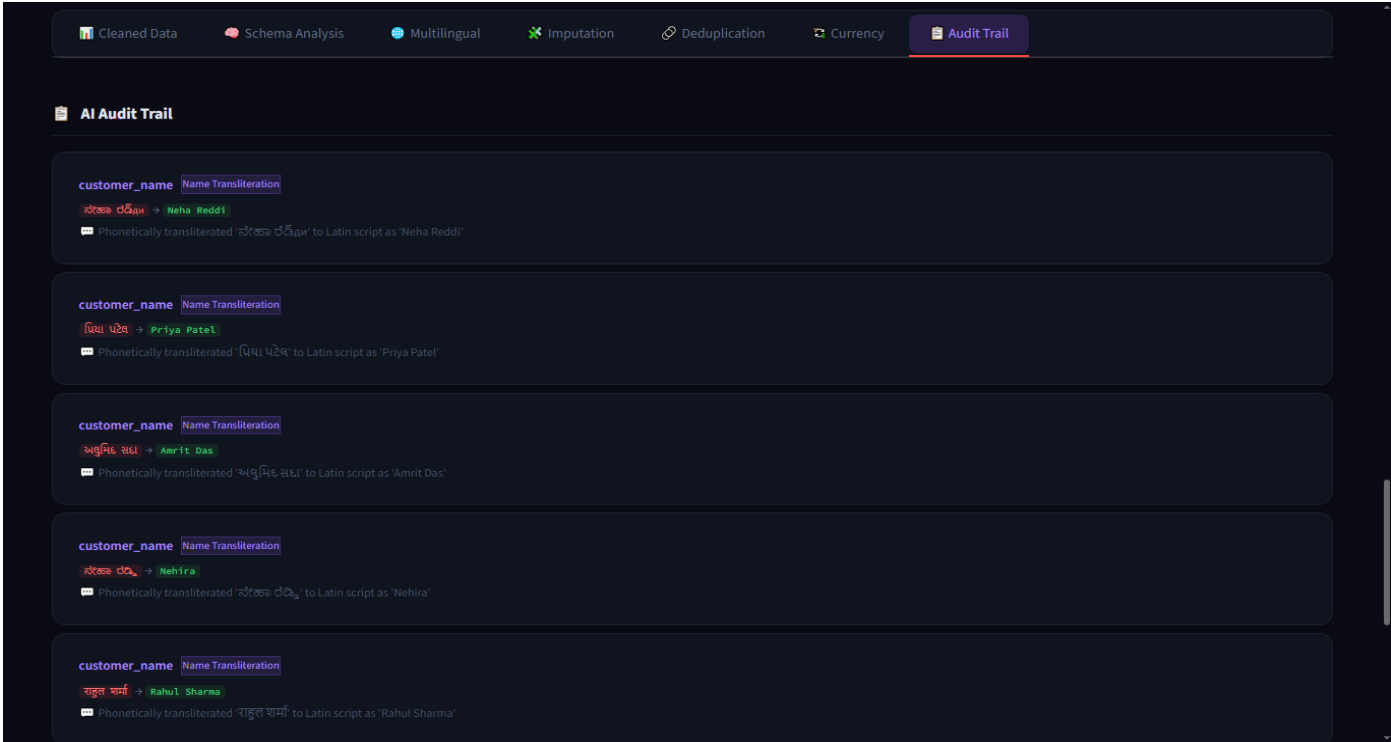


Figure. 9.15 AI Audit Trail.

Interface note	Details
What this screen shows	The audit trail cards display field-level changes such as transliteration of customer_name values, the source value, and the resulting Latin-script representation.
Visible UI elements	Stacked audit cards, field labels, old and new values, and short explanation text under each change.
Why it matters	This is critical for trust, especially when business users need to review or defend how a value was altered.
Backend mapping	agents/explanation_agent.py and backend/pipeline.py create the metadata that powers the human-readable audit trail.
Next step	The cleaned CSV, Excel export, and report download actions can now be used with confidence.

13. Mode 2: Live Database Studio

Mode 2 extends the same cleaning intelligence to connected SQL sources. The user can connect to a database, browse tables, ask questions in plain English, and run the AI cleaning pipeline directly on live records. This workflow is especially valuable for operational teams because it avoids manual export steps and keeps the database context visible at every stage. The screenshots below follow the live studio from connection through audit logging.

10.1 Live Database Studio Landing View

This is the entry screen for the live database workflow. It shows the Universal Database Cleaning and Query Studio as a connected workspace instead of a file upload page.

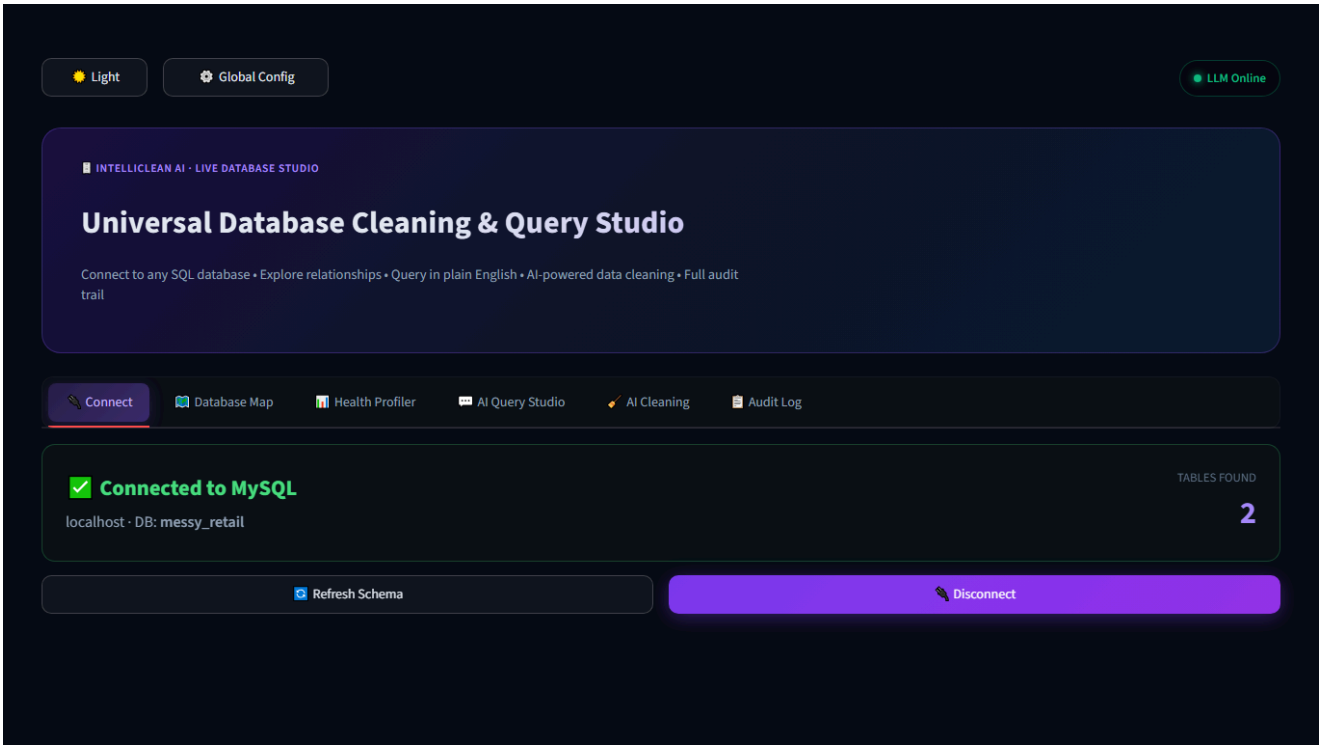


Figure. 10.1 Live Database Studio Landing View.

Interface note	Details
What this screen shows	The interface shows the main navigation tabs for Connect, Database Map, Health Profiler, AI Query Studio, AI Cleaning, and Audit Log, plus the current connection status.
Visible UI elements	Connection-ready hero banner, tab navigation, live connection summary, refresh and disconnect controls, and the table count indicator.
Why it matters	It confirms that IntelliClean can move beyond spreadsheets and operate directly on SQL-backed data sources.
Backend mapping	pages/Live_Database.py coordinates the dashboard and stores the live connection engine in session state.
Next step	The user either connects to a database or switches into the database map and cleaning tabs.

10.2 Database Connection and Live Status

This view shows the connected state of the database studio. It demonstrates that the application has successfully reached a live SQL database and can now inspect its contents.

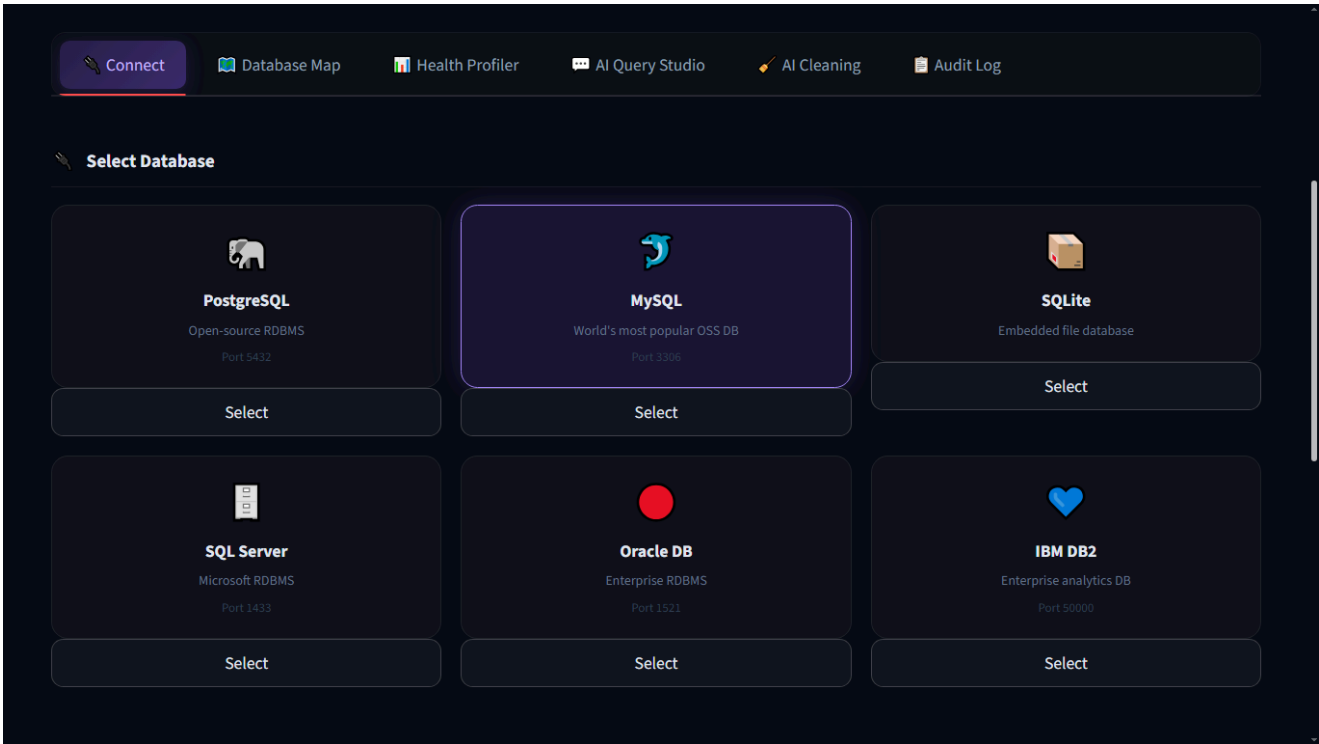


Figure. 10.2 Database Connection and Live Status.

Interface note	Details
What this screen shows	The dashboard presents the active connection, the available tables, and the studio cards that let the user navigate the live database workflow.
Visible UI elements	Connected status card, table list, database summary tiles, and the same top-level tab navigation used across the live studio.
Why it matters	Connection confirmation is essential because all remaining actions depend on a valid live database session.
Backend mapping	backend/db_connector.py creates the SQLAlchemy connection and exposes the connection details to the Streamlit UI.
Next step	The user can open the connection form or proceed to database map and profiling actions.

10.3 Connection Form and Database Credentials

This screen captures the credential-entry form used to connect to a database. It makes the connection process explicit and user-guided.

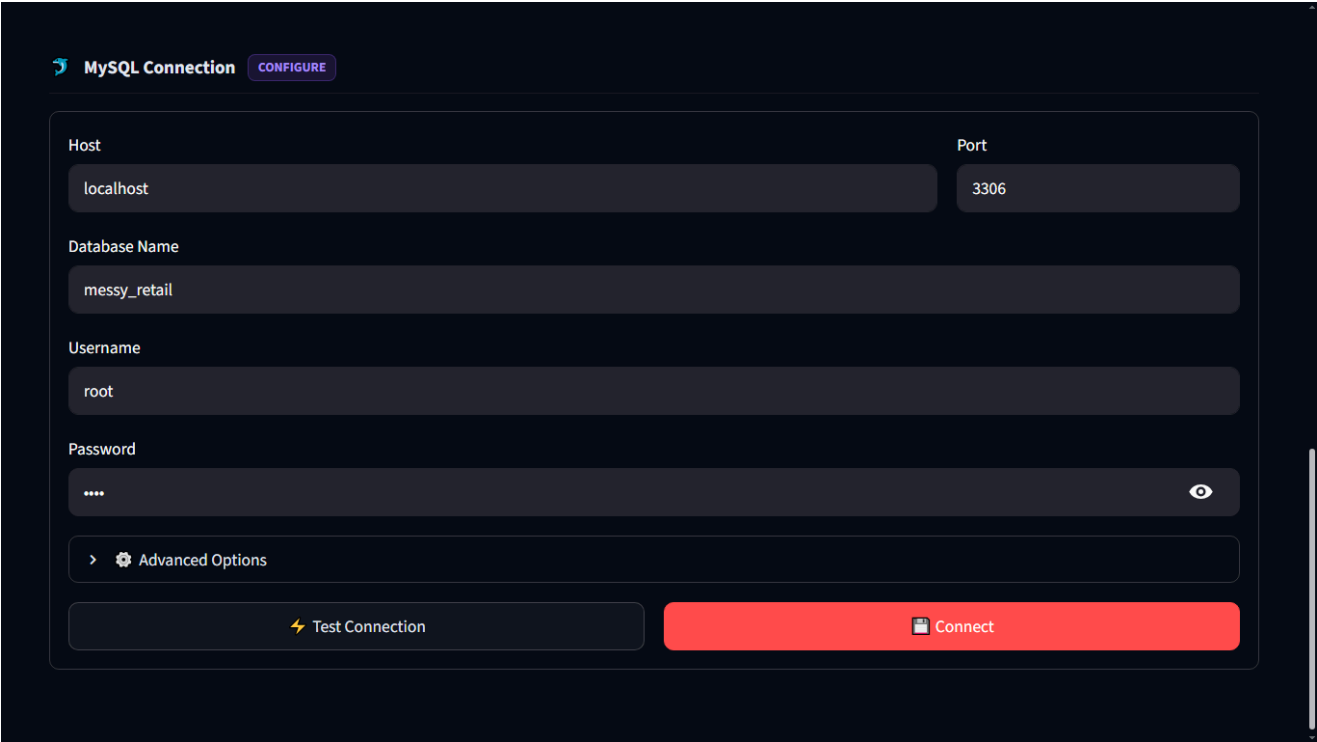


Figure. 10.3 Connection Form and Database Credentials.

Interface note	Details
What this screen shows	The form collects host, port, username, password, and database name before the connection is opened.
Visible UI elements	Input fields for database credentials, form layout, and a prominent connect action button.
Why it matters	It provides a simple and repeatable way to connect to different SQL sources without editing code.
Backend mapping	pages/Live_Database.py submits the credentials to backend/db_connector.py, which then builds the engine and inspects the schema.
Next step	After connection, the user can refresh the schema, map tables, or open the health profiler.

10.4 Connection Controls and Workspace Actions

This view shows the database workspace after connection controls are available. It highlights the operational buttons that keep the live session manageable.

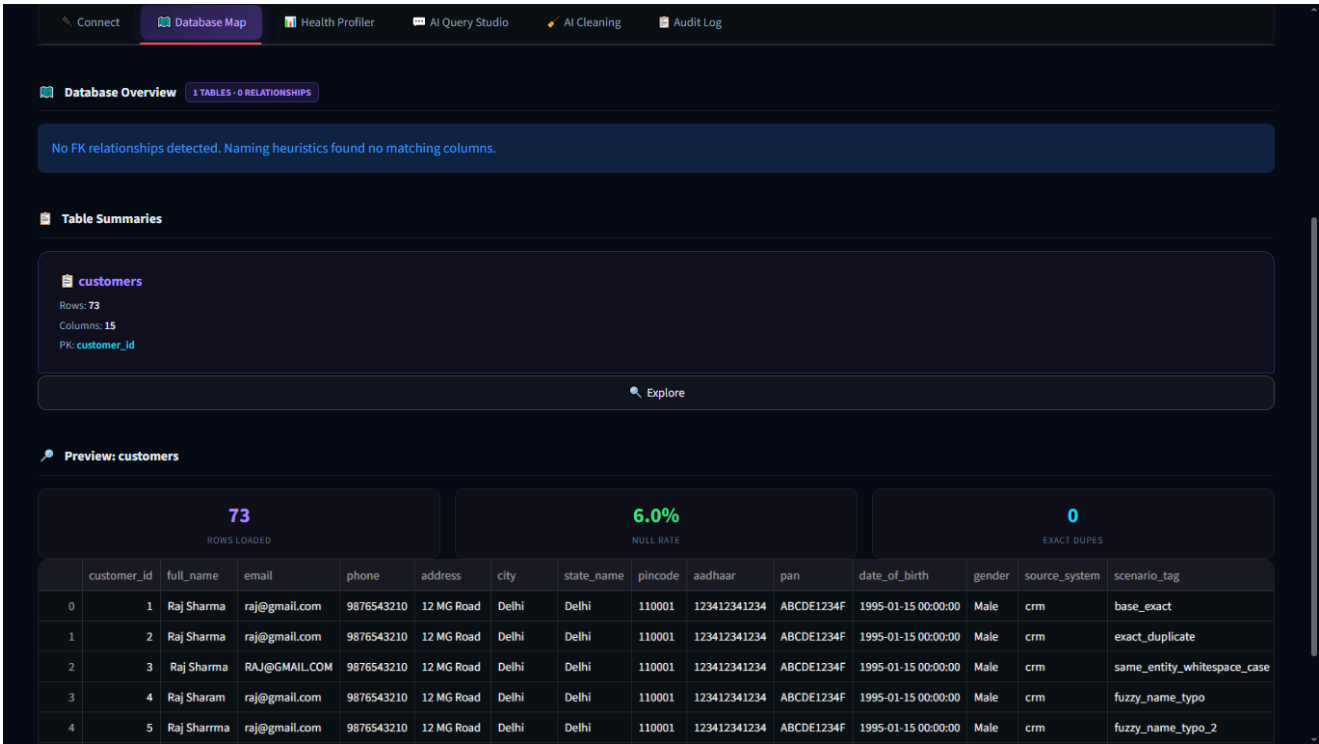


Figure. 10.4 Connection Controls and Workspace Actions.

Interface note	Details
What this screen shows	The page presents the connected workspace area with action controls for managing the live session and moving between studio tasks.
Visible UI elements	Connection actions, workspace container, and a database-oriented control layout.
Why it matters	Having clear connection controls prevents users from losing their session or accidentally working on the wrong source.
Backend mapping	The live connection state is maintained in Streamlit session state while the SQLAlchemy engine remains active.
Next step	The user can move to table mapping, profiling, or AI-based cleaning.

10.5 Health Profiler Entry

This is the health-profiler launch panel for live data. It prepares the user to run a structured scan of the selected table.

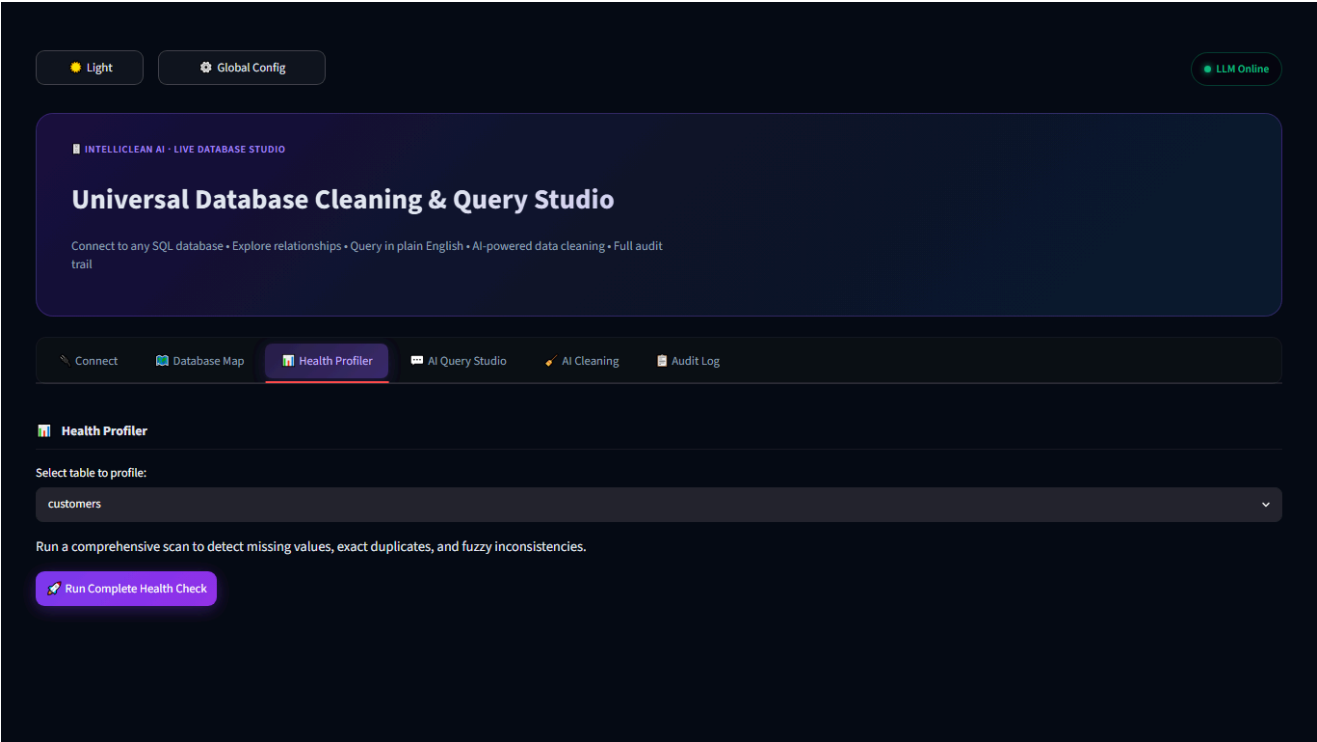


Figure. 10.5 Health Profiler Entry.

Interface note	Details
What this screen shows	The interface shows the table selector and the button that launches a complete health check on the live dataset.
Visible UI elements	Database studio tabs, table dropdown, short help text, and the Run Complete Health Check button.
Why it matters	It mirrors the file-based profiling experience, but now works directly against the connected SQL table.
Backend mapping	backend/profiler.py collects table-level metrics while the live connection is supplied by backend/db_connector.py.
Next step	The user runs the health check to inspect missing values, duplicates, and statistical quality.

10.6 Database Health Profiler Summary View

This view reports the live profiling results after the health check runs. It makes the table quality visible through summary cards and counts.

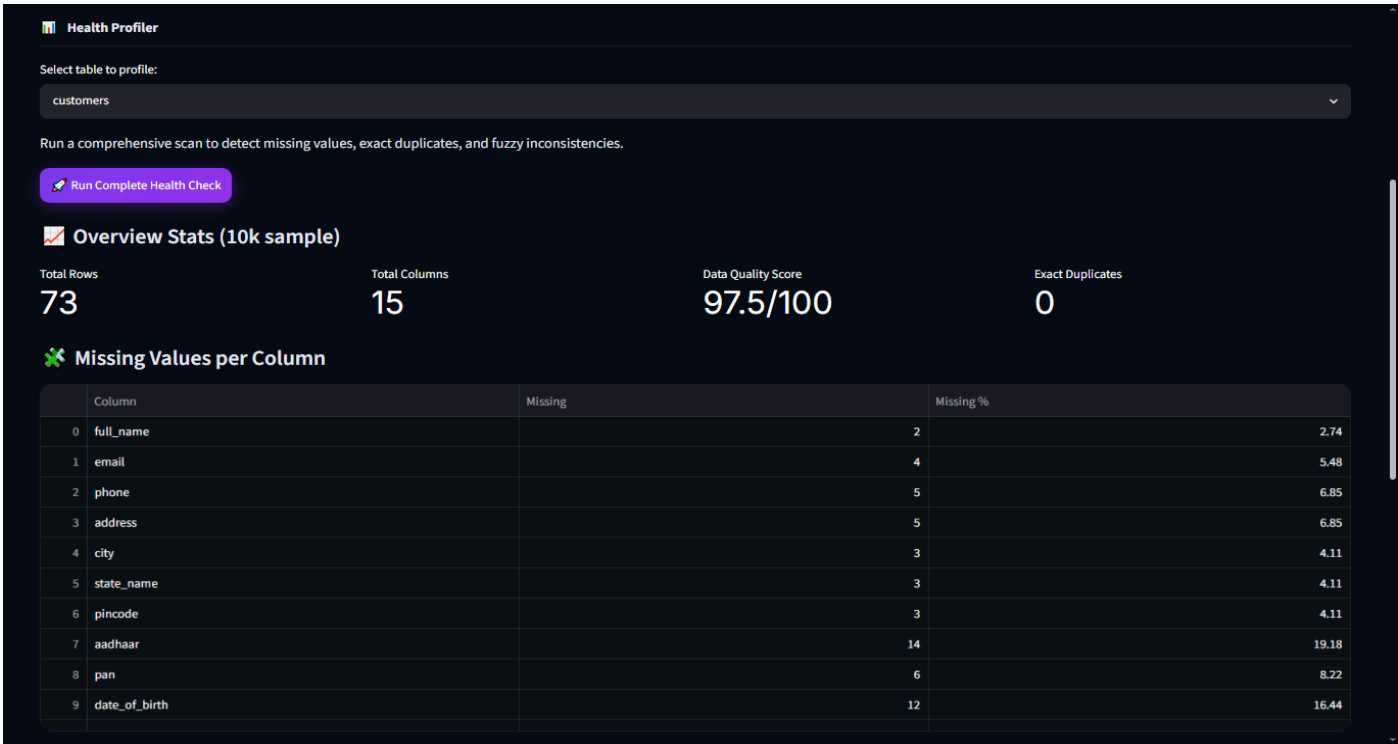


Figure. 10.6 Health Profiler Summary View.

Interface note	Details
What this screen shows	The screen shows row count, column count, missing-value total, duplicate count, and the profiled table preview with the connected data.
Visible UI elements	Profiling cards, numeric metrics, table preview, and the health profiler tab highlighted in the navigation.
Why it matters	It gives the user a fast quality snapshot of the live database before any cleaning is applied.
Backend mapping	backend/profiler.py and the live table fetch logic generate the statistics shown here.
Next step	The user can inspect deeper profiling details or launch AI cleaning for that table.

10.7 Health Profiler Detailed View

This screen provides a more detailed live health profile. It exposes the table scan results in a compact but data-rich layout.

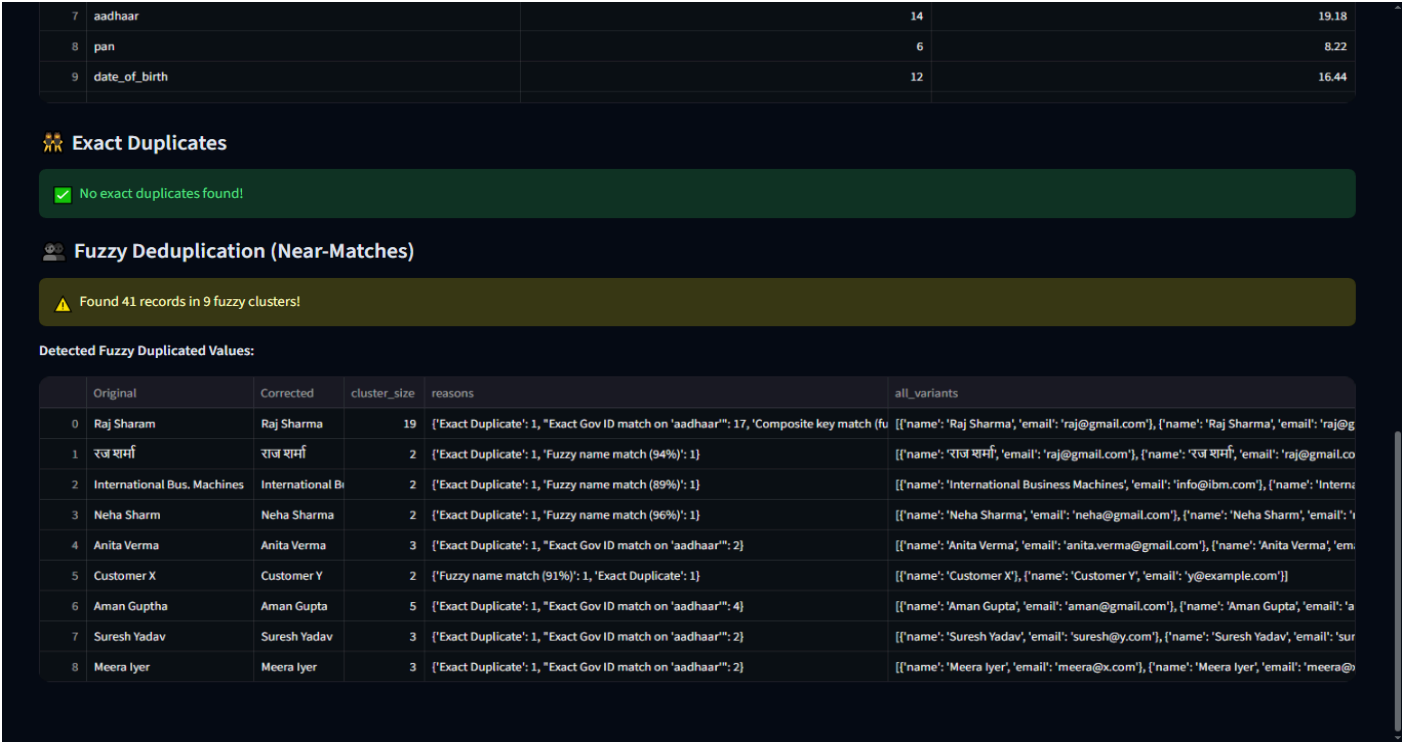


Figure. 10.7 Health Profiler Detailed View.

Interface note	Details
What this screen shows	The page shows the same profiling context with a richer table of values, metrics, and live quality indicators.
Visible UI elements	Metric cards, table-level summary, and the detailed profiling result list.
Why it matters	A detailed health profile helps analysts understand whether the connected table is ready for AI cleaning or manual review.
Backend mapping	The result is based on the same live profiling engine but presented in a denser UI layout for review.
Next step	The user can move to the AI Query Studio or into the AI cleaning workspace.

10.8 AI Query Studio

This is the natural-language query interface for live databases. It lets users ask questions in plain English and receive results from SQL execution.

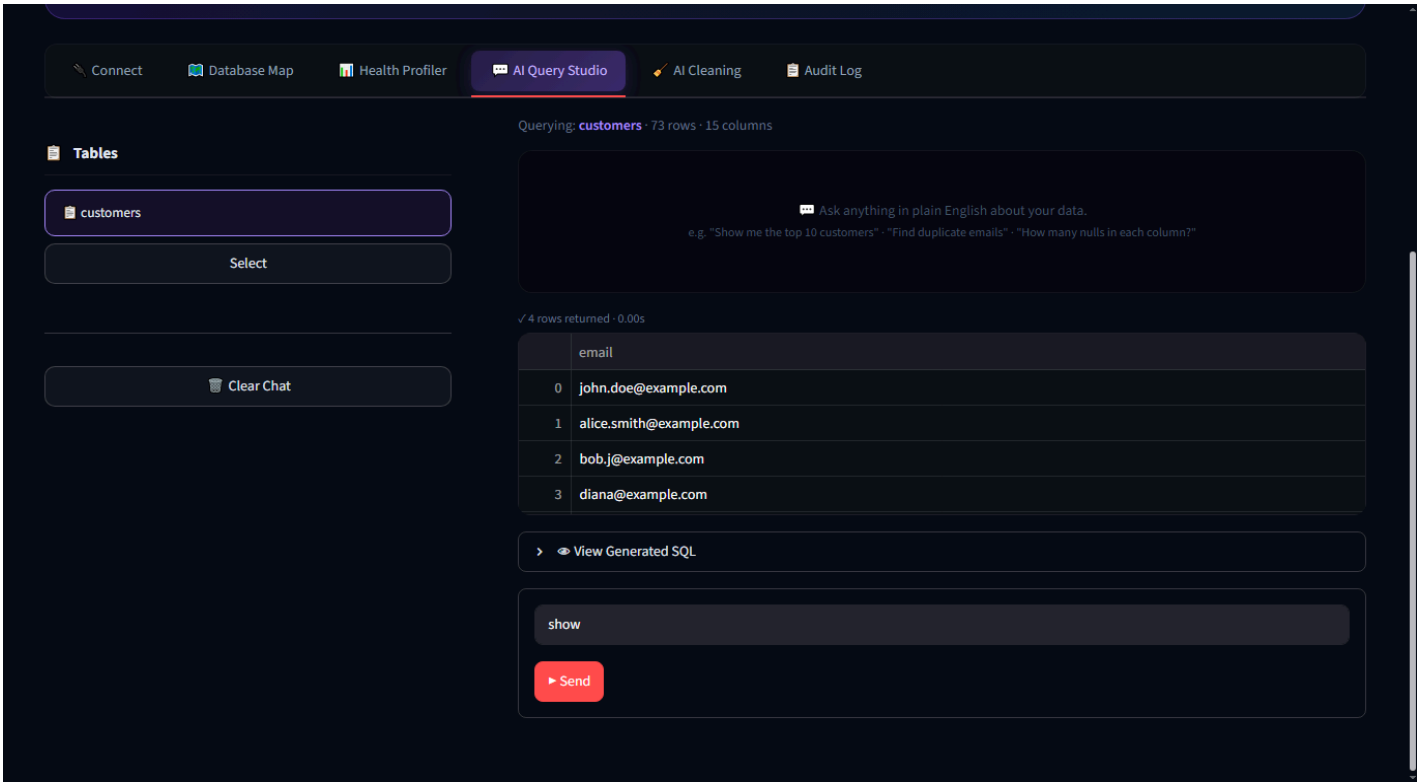


Figure. 10.8 AI Query Studio.

Interface note	Details
What this screen shows	The screen shows a table selector, a question box, result preview, a view-generated-SQL toggle, and a send button for running the query.
Visible UI elements	Selected table, prompt area, result table, SQL reveal section, and clear-chat control.
Why it matters	It lowers the barrier to database exploration for non-SQL users while keeping execution restricted to safe read-only operations.
Backend mapping	agents/nl_query_agent.py generates SQL from the prompt, and backend/db_connector.py executes the resulting read-only query.
Next step	The user can inspect the generated SQL or ask another question about the connected table.

10.9 Audit Log Overview

This view records the actions and outputs generated during the live studio workflow. It acts as the first layer of operational traceability.

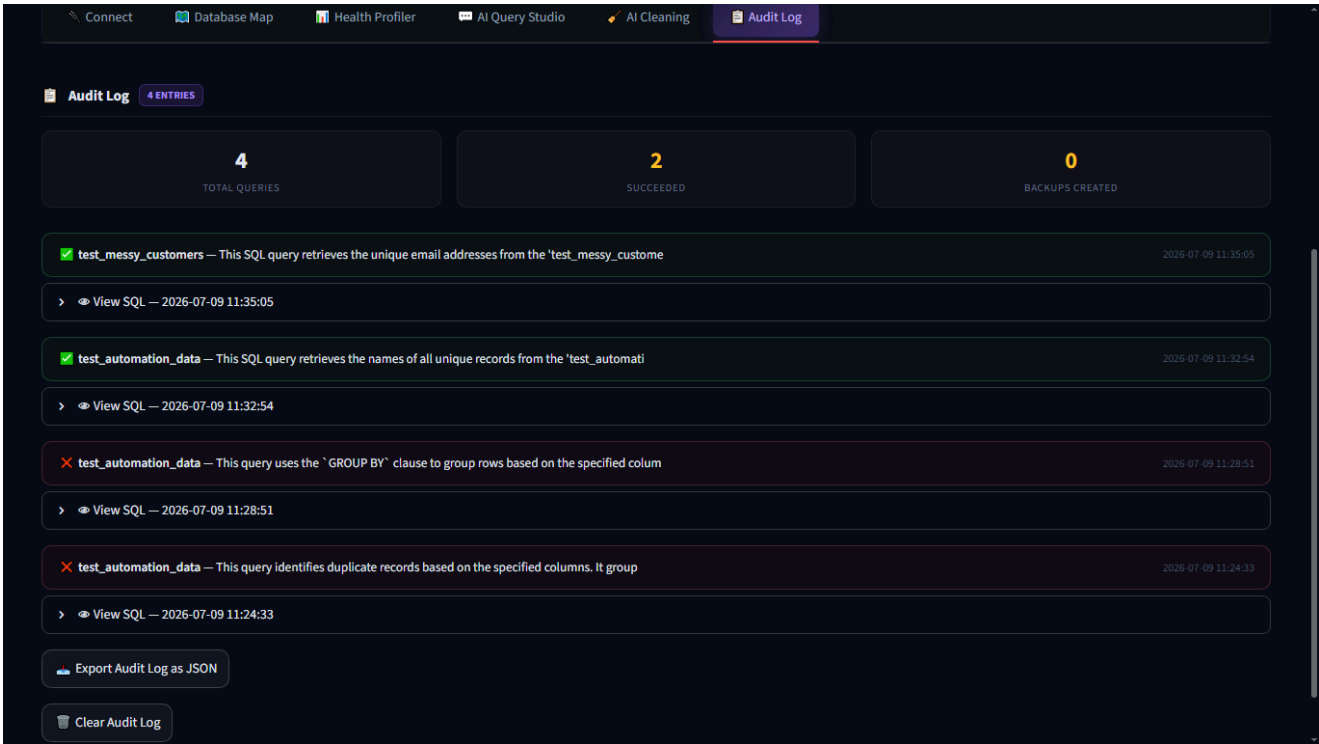


Figure. 10.9 Audit Log Overview.

Interface note	Details
What this screen shows	The page shows a log feed with color-coded messages, summarizing recent events and system actions for the live database session.
Visible UI elements	Audit messages, log styling, status cues, and the live workflow navigation bar.
Why it matters	An explicit log view helps users and reviewers understand exactly what the database studio did during the session.
Backend mapping	backend/pipeline.py, pages/Live_Database.py, and the audit metadata collection all feed this view.
Next step	The user can expand the log or proceed to the cleaning workspace for live data transformation.

10.10 Audit Log - Expanded View

This is a more detailed audit-log capture from the live workflow. It shows how the audit trail can be expanded to include richer event information.

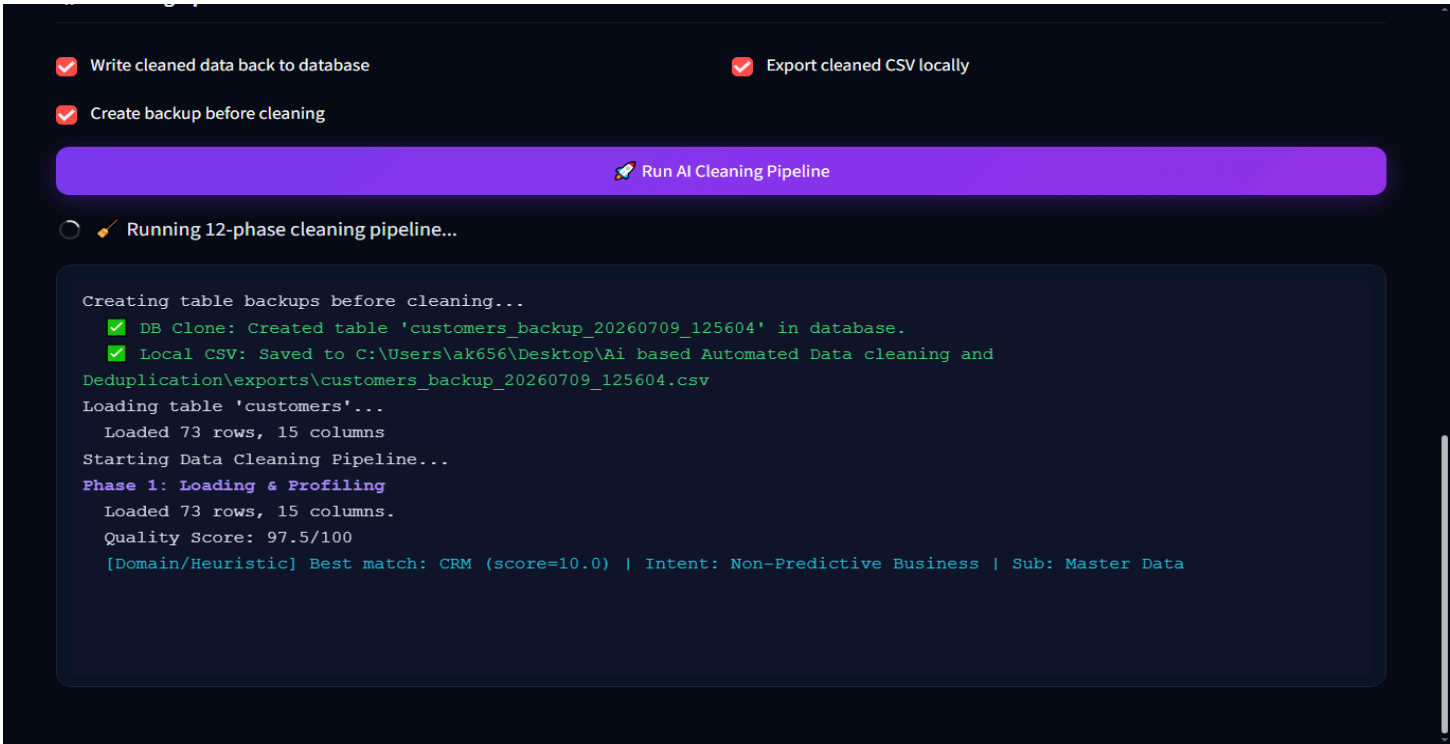


Figure. 10.10 Audit Log - Expanded View.

Interface note	Details
What this screen shows	The panel displays a longer sequence of log lines and status text, making it easier to trace the session history.
Visible UI elements	Expanded log entries, green success cues, and clean separation between event groups.
Why it matters	A longer log view is useful for troubleshooting and for proving what the system did at each stage.
Backend mapping	The log contents come from the same metadata and callback patterns used throughout the pipeline.
Next step	The user can move to the AI Cleaning workspace after verifying the session history.

10.11 AI Cleaning Studio - Summary

This screen shows the live AI cleaning workspace before execution. It brings together the selected table and a pre-cleaning profile so the user can review the input state first.

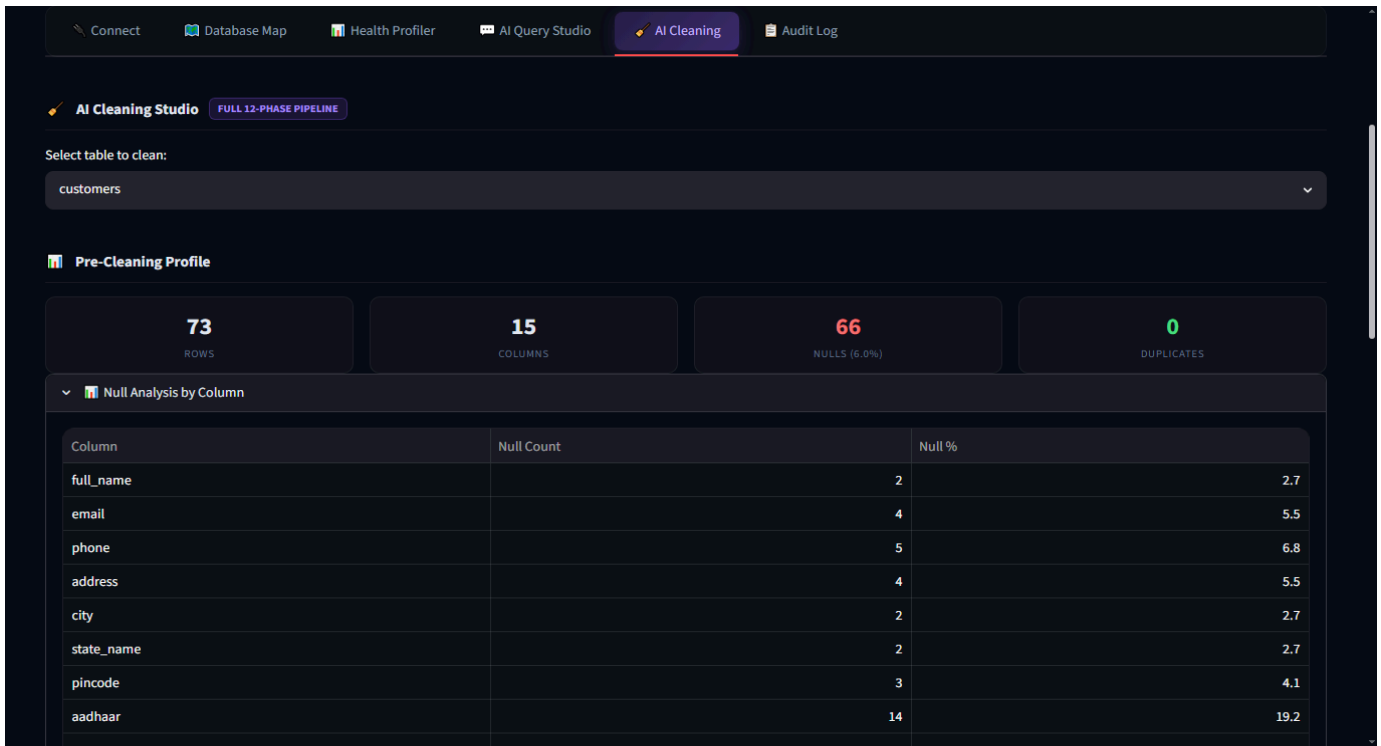


Figure. 10.11 AI Cleaning Studio - Summary.

Interface note	Details
What this screen shows	The page shows the chosen table, the pre-cleaning metrics, and a null-analysis section that lists the columns and missing percentages.
Visible UI elements	Table selector, row/column/null summary cards, and the null-analysis table by column.
Why it matters	This is the starting point for applying the 12-phase pipeline directly to database records.
Backend mapping	pages/Live_Database.py and backend/profiler.py prepare the live cleaning context, then backend/pipeline.py performs the transformations.
Next step	The user can launch the cleaning run or inspect the second AI Cleaning Studio capture.

10.12 AI Cleaning Studio - Detailed View

This second AI Cleaning Studio capture shows the same live cleaning environment from a slightly different point in the scroll position.



Figure. 10.12 AI Cleaning Studio - Detailed View.

Interface note	Details
What this screen shows	The table and summary cards remain visible, but the page emphasizes the database rows, columns, and null distribution inside the workspace.
Visible UI elements	Live summary metrics, table context, and the detailed null-analysis area.
Why it matters	It reinforces that the live cleaning workflow is interactive and can be inspected before the final output is committed.
Backend mapping	The same live cleaning orchestration and profiling code drives this state of the interface.
Next step	The user can continue into the cleaned-data output, schema analysis, multilingual processing, or deduplication tabs.

10.13 Cleaned Live Data Preview

This screen shows the output of the live cleaning process after the database table has been transformed.

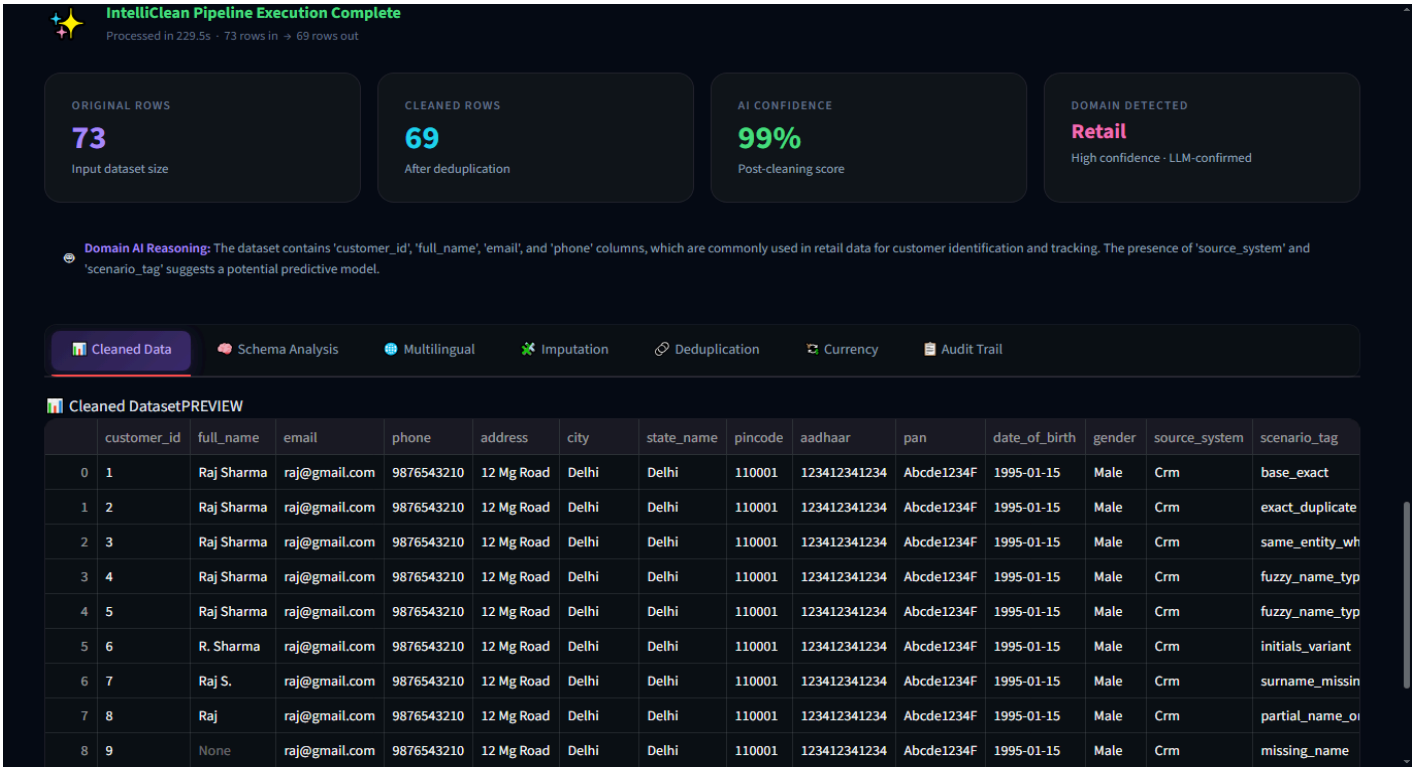


Figure. 10.13 Cleaned Live Data Preview.

Interface note	Details
What this screen shows	The page presents the cleaned records together with summary metrics that show the effect of the pipeline on the live table.
Visible UI elements	Cleaned data table, updated profiling cards, and the same studio navigation that lets the user move across analysis tabs.
Why it matters	The cleaned preview is the proof that the live workflow produced a usable result rather than just logs.
Backend mapping	backend/exporter.py and backend/pipeline.py produce the final DataFrame and metadata for the UI.
Next step	The user can inspect schema analysis, multilingual handling, imputations, or deduplication.

10.14 Live Schema Analysis

This screen shows the semantic interpretation of the live table columns. It is the database equivalent of the spreadsheet schema analysis view.

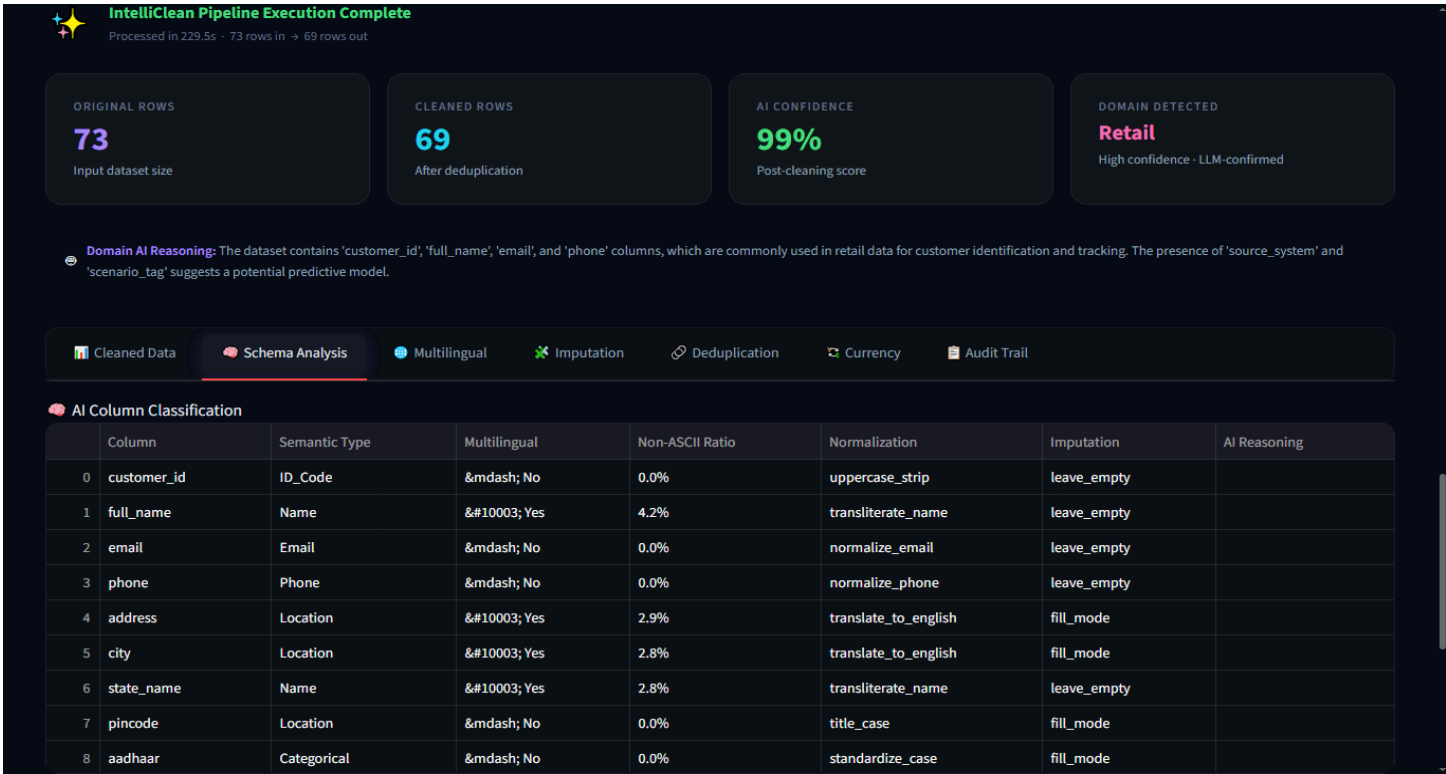


Figure. 10.14 Live Schema Analysis.

Interface note	Details
What this screen shows	The table classifies columns, documents multilingual behavior, and ties each field to a normalization and imputation strategy.
Visible UI elements	Semantic type table, navigation tabs, and the live database context around the analysis panel.
Why it matters	Knowing the meaning of a column is essential before attempting translation, deduplication, or imputation.
Backend mapping	backend/schema_detector.py and agents/schema_agent.py classify the fields while the domain profiler supplies context.
Next step	The user can open the multilingual, imputation, or deduplication tabs for deeper cleaning controls.

10.15 Live Multilingual Processing

This page focuses on live multilingual detection and normalization. It exposes how non-ASCII records are handled after the schema stage.

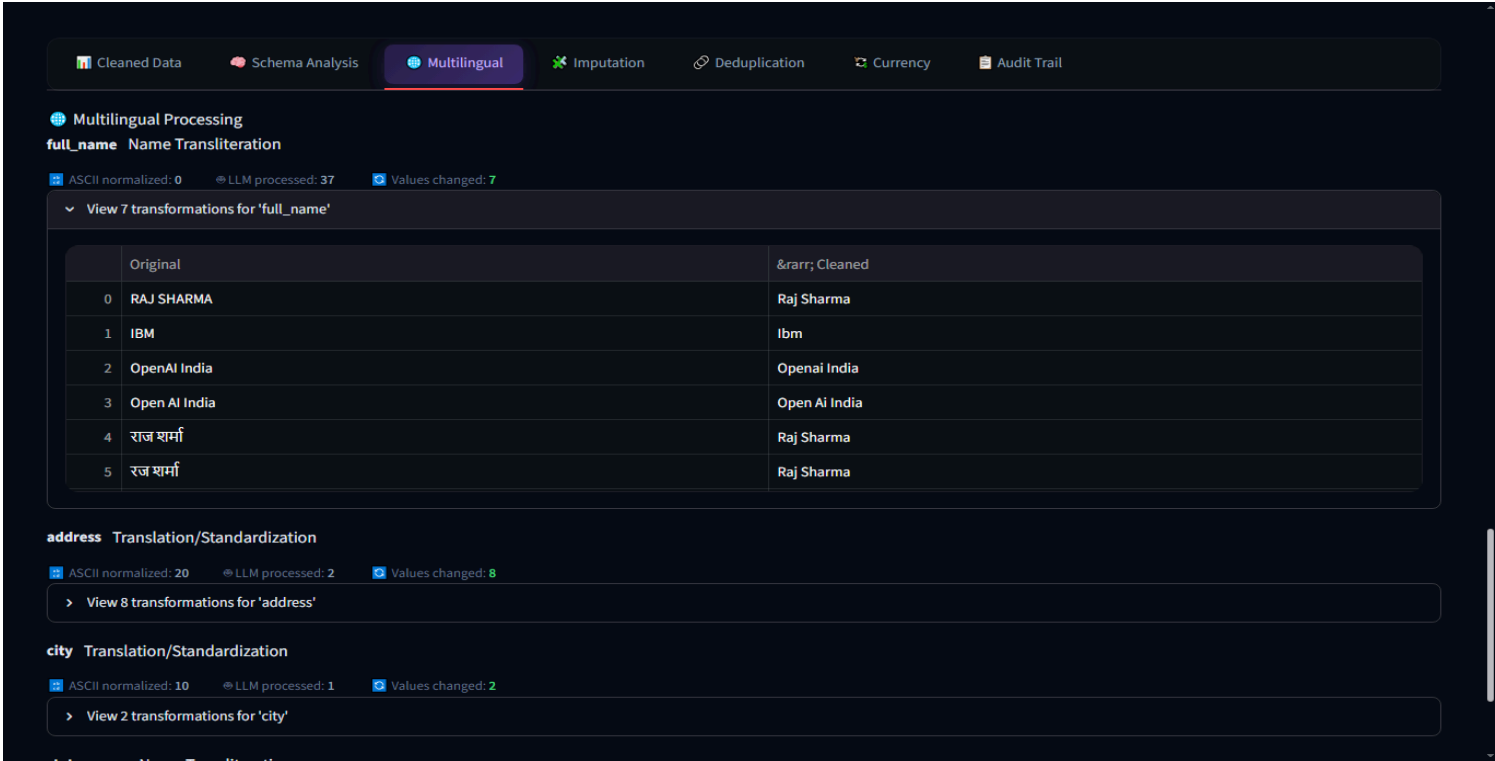


Figure. 10.15 Live Multilingual Processing.

Interface note	Details
What this screen shows	The view lists the multilingual processing panel and highlights how many values were normalized or left untouched for the selected field.
Visible UI elements	Multilingual processing card, field-level summary, and the same live navigation tabs.
Why it matters	It demonstrates that the live database workflow preserves identity fields while still normalizing non-English records for downstream matching.
Backend mapping	cleaning/multilingual.py records the transliteration or translation work in the metadata used by the UI.
Next step	The user can move to imputation, deduplication, or audit trails.

10.16 Live Imputation View

This screen shows how the live workflow fills missing values. It reveals the column-level imputation logic in the database studio.

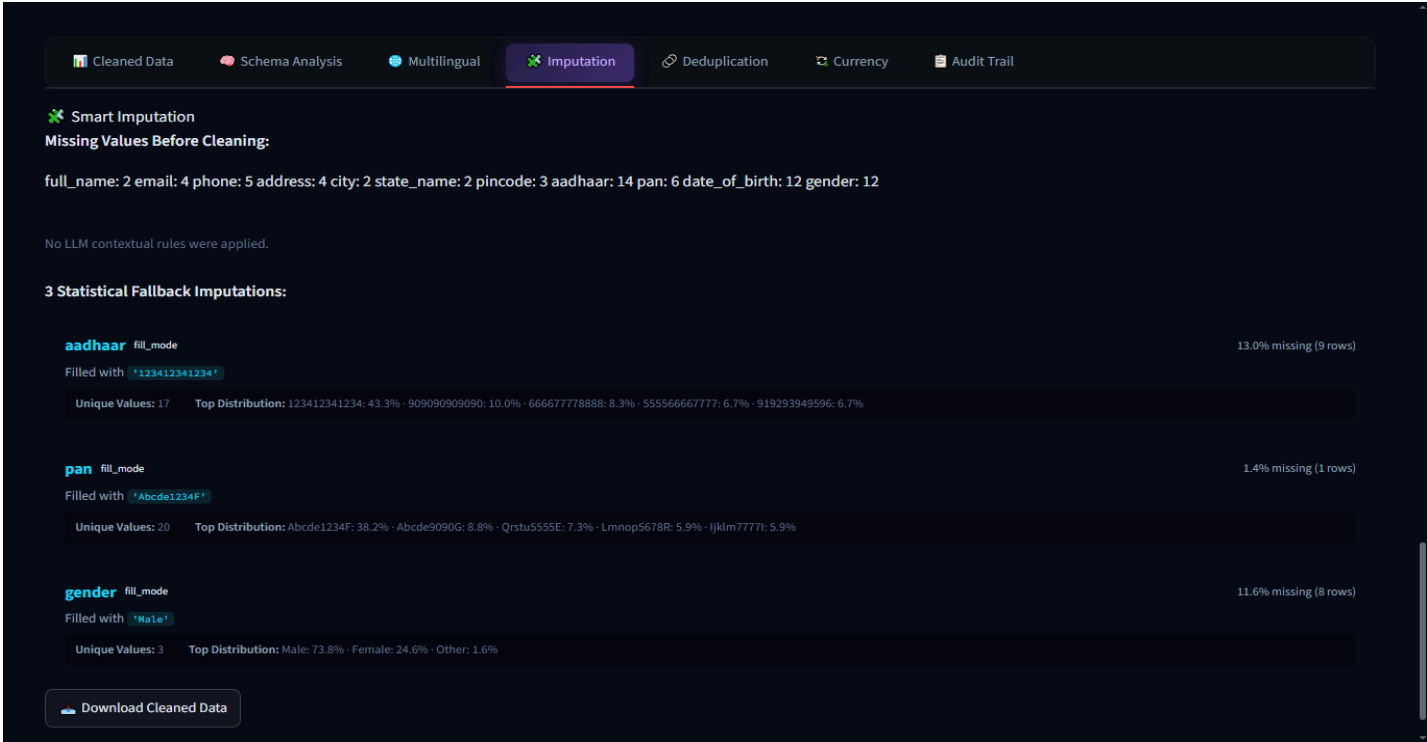


Figure. 10.16 Live Imputation View.

Interface note	Details
What this screen shows	The page presents the imputation result cards and the columns that were filled or left untouched during live cleaning.
Visible UI elements	Imputation panel, field names, fill labels, and result indicators for live data repair.
Why it matters	It provides transparency into how the system handled gaps in the connected database.
Backend mapping	agents/planner_agent.py proposes contextual rules and cleaning/missing_values.py applies the actual fill operations.
Next step	The user can review deduplication or check the audit trail to confirm the changes.

10.17 Live Deduplication View

This screen displays deduplication for live database records. It makes the identity-resolution step visible in the database studio.

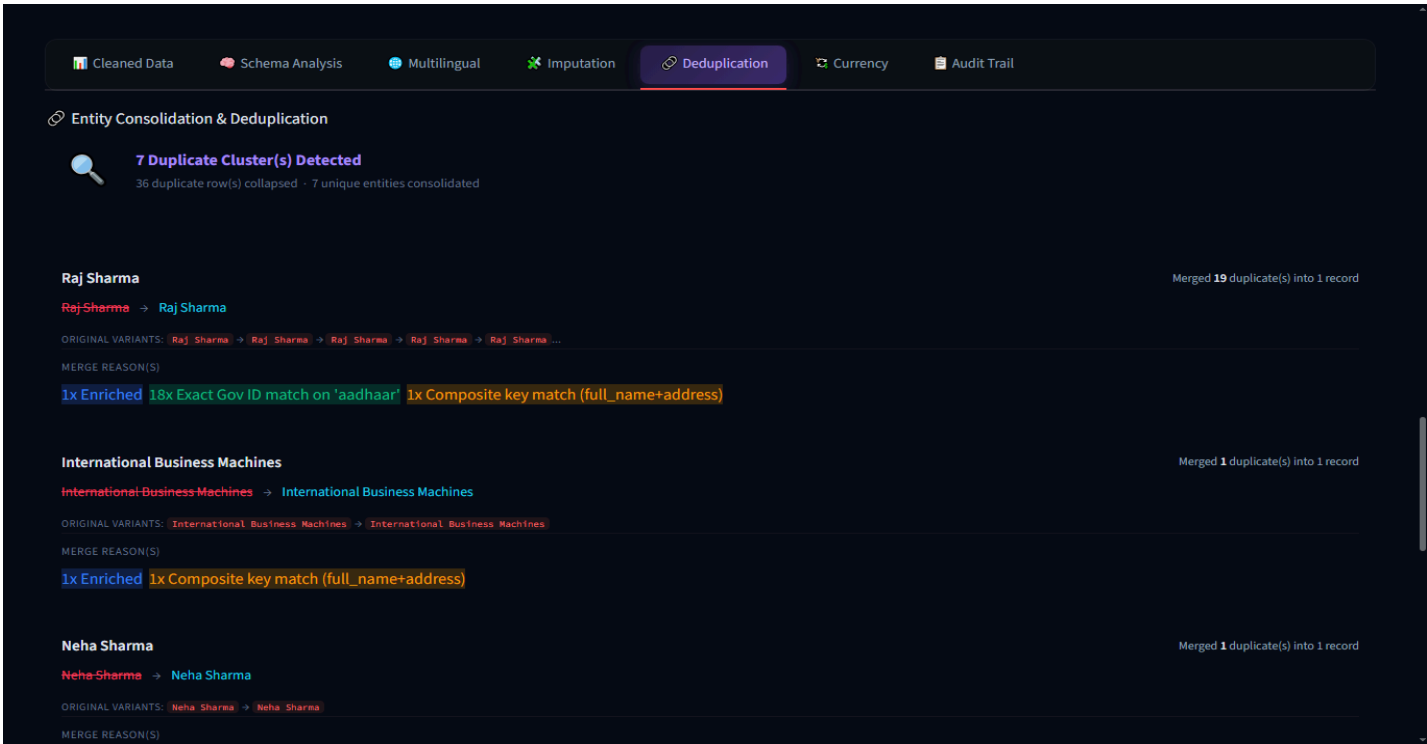


Figure. 10.17 Live Deduplication View.

Interface note	Details
What this screen shows	The page shows deduplication cards, cluster summaries, and the records that were merged or collapsed during the run.
Visible UI elements	Deduplication cards, highlighted matches, and a navigation bar linking the stage to the rest of the pipeline.
Why it matters	Live deduplication is crucial because many database tables accumulate repeated customer or transaction records over time.
Backend mapping	cleaning/deduplication.py and cleaning/entity_resolution.py work together to detect and merge clusters.
Next step	The user can inspect the audit trail to see what was changed and why.

10.18 Live Audit Trails

This final live-workflow view captures the complete audit trail. It is the evidence layer for the database cleaning session.

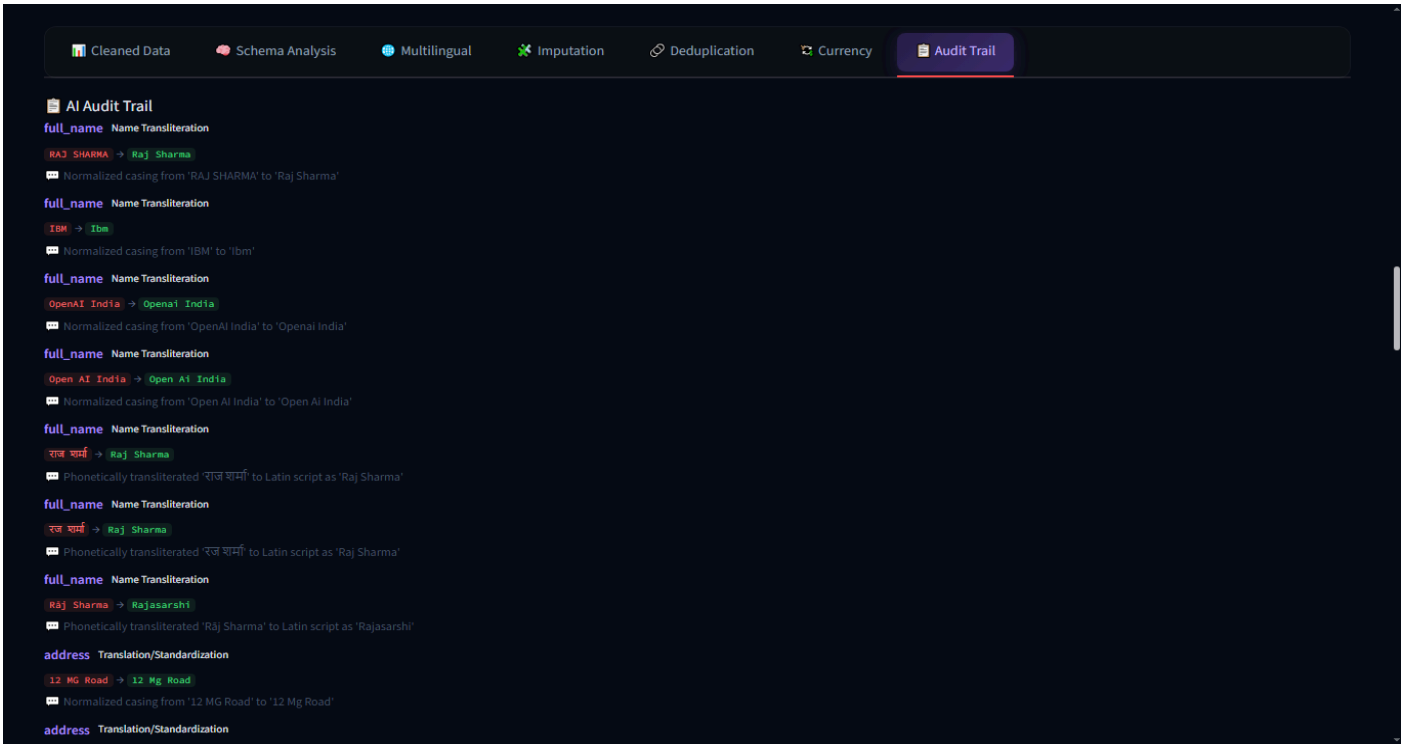


Figure. 10.18 Live Audit Trails.

Interface note	Details
What this screen shows	The panel lists the transformation history so the user can review which values changed, which stage made the change, and the reason attached to each edit.
Visible UI elements	Audit trail entries, status markers, and the final database workflow context.
Why it matters	It closes the loop for governance and makes the live workflow suitable for review, debugging, and reporting.
Backend mapping	agents/explanation_agent.py and the pipeline metadata construct the final human-readable audit output.
Next step	The live data session is complete, and the user can export or reuse the cleaned result.

14. Backend Module Reference and AI Agents

The provided markdown guides describe a modular backend in which each file has a clear job. This section groups the modules by responsibility so the system can be understood as a complete engineering stack rather than a loose set of scripts.

The tables below summarize the main UI/orchestration layer, the semantic and validation agent layer, and the deterministic cleaning layer.

Module	Role	Key behavior
app.py	Main Streamlit orchestrator	Manages session state, streams pipeline logs into the UI, and renders the final analytics panels.
pages/Live_Database.py	Live DB workspace	Handles connection forms, table discovery, NL query execution, and live cleaning actions.
backend/pipeline.py	Central orchestrator	Runs the ordered cleaning phases and keeps metadata synchronized across the pipeline.
backend/exporter.py	Output manager	Writes cleaned CSV/Excel outputs and saves the generated report text.
agents/llm_client.py	LLM transport	Discovers the active local model, retries requests, and enforces JSON output.
agents/explanation_agent.py	Report writer	Turns metadata into a readable markdown explanation for business users.
agents/nl_query_agent.py	Text-to-SQL agent	Generates executable SQL from plain English and corrects query errors when needed.

Module	Role	Key behavior
agents/schema_agent.py	Semantic profiler	Assigns meaning such as email, person name, currency, date, or free text.
backend/schema_detector.py	Heuristic pre-filter	Fast pre-classification that reduces the need for LLM calls on obvious columns.
backend/domain_profiler.py	Industry detector	Infers the business domain using keywords and a compact LLM confirmation step.
agents/planner_agent.py	Imputation strategist	Generates context-aware fill rules based on domain, schema, and profile signals.
agents/validation_agent.py	Cognitive auditor	Checks the cleaned output for semantic issues that simple rules may miss.
backend/validator.py	Math and rule checker	Performs hard validations such as negative value checks and consistency tests.
backend/profiler.py	Statistical baseline	Computes the initial quality score, missingness, duplicates, and memory usage.

Module	Role	Key behavior
backend/loader.py	Universal loader	Reads CSV/XLS/XLSX data, normalizes missing markers, and audits memory usage.
cleaning/quality_filter.py	Junk scrubber	Drops rows or columns that are too sparse or obviously useless.
cleaning/string_cleaner.py	Invisible character remover	Removes zero-width, non-breaking, and extra whitespace characters.
cleaning/standardizer.py	Semantic formatter	Standardizes emails, names, locations, and phone-like strings.
cleaning/datatype_cleaner.py	Datatype fixer	Converts mixed-type columns, booleans, and date-like strings into clean types.

cleaning/currency_converter.py	Currency normalizer	Extracts symbols, strips noise, and converts mixed currencies into a common numeric scale.
cleaning/multilingual.py	Translation engine	Translates unique non-ASCII free-text values with caching and transliteration fallback.
cleaning/deduplication.py	Identity resolver	Uses blocking and fuzzy matching to detect exact, partial, fuzzy, and semantic duplicates.
cleaning/entity_resolution.py	Cluster merger	Builds a golden record by backfilling the most complete row in each duplicate cluster.
cleaning/missing_values.py	Smart imputer	Applies rule-based fills first, then statistical mean/median/mode fallback.
cleaning/outliers.py	Anomaly handler	Detects numeric outliers with IQR and clips them instead of dropping rows.

15. Functional Requirements and Design Constraints

The requirements document emphasizes coverage across file ingestion, live databases, AI-assisted schema understanding, deduplication, explainability, and export. It also makes the performance and privacy goals explicit.

Area	Requirement
File handling	Support CSV, XLS, and XLSX uploads with automatic delimiter and sheet handling.
Database support	Connect to major SQL databases and sample large tables safely to avoid memory issues.
Profiling	Report row count, column count, missingness, duplicates, unique counts, and memory usage.
Schema understanding	Classify columns into entity, identifier, numeric, temporal, free text, and categorical groups.
Duplicate detection	Support exact, partial, fuzzy, semantic, and cross-language duplicate detection.
NL query engine	Convert plain English into safe, read-only SQL and allow self-correction after errors.
Explainability	Record original value, transformed value, reason, confidence, and responsible agent.
Non-functional	Run locally, use limited memory, avoid $O(N^2)$ comparisons, and keep inference offline.

The non-functional requirements are equally important: the system should remain usable on modest hardware, use blocking before fuzzy comparisons, prefer pyarrow and downcasting for memory efficiency, and keep LLM prompts small enough to avoid timeouts.

16. Deployment and Operations Guide

Ubuntu Server Deployment Guide for IntelliClean AI

This guide provides a complete step-by-step path to deploy IntelliClean AI on a production Ubuntu server using a local model server, a Python virtual environment, and a Streamlit service.

System Prerequisites

Operating System: Ubuntu Server 20.04 LTS, 22.04 LTS, or 24.04 LTS.

Python Version: Python 3.10 or higher. Python 3.12 is recommended.

System Hardware: minimum 12 GB RAM. 16 GB+ RAM or a CUDA-capable GPU is recommended for smoother local inference.

Network Open Ports: 8501 for Streamlit. 1234 for LM Studio server access, which should remain local-only.

1. Clone the Repository & Configure Workspace

```
git clone https://github.com/ashudsvv99/Ai-based-Data-Cleaning-and-deduplications-tool.git
cd Ai-based-Data-Cleaning-and-deduplications-tool
```

2. Set Up the Local LLM Inference Server (LM Studio CLI)

Install the CLI daemon:

```
curl -fsSL https://lmstudio.ai/install.sh | bash
```

Add the LM Studio bin folder to your shell path:

```
export PATH="$HOME/.lmstudio/bin:$PATH"
echo 'export PATH="$HOME/.lmstudio/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Start the daemon, download the model, load it, and then launch the local OpenAI-compatible API server on port 1234.

```
lms daemon up
lms get google/gemma-4-e2b
lms load google/gemma-4-e2b
lms server start
```

Do not run `sudo lms`. The CLI is installed for the current user, so `sudo` will fail with a command-not-found error.

3. Set Up Python Virtual Environment & Install Dependencies

```
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
```

4. Launch the Streamlit Web Application

Always launch the application with `streamlit run app.py`. Running the file directly with `python3 app.py` or `./app.py` breaks Streamlit session state.

```
streamlit run app.py
```

For a persistent production process, use `nohup`, `tmux`, or a `systemd` service. A `systemd` unit is the most robust option because it restarts on crash and starts on boot.

```

sudo nano /etc/systemd/system/intelliclean.service

[Unit]
Description=IntelliClean AI Web Application
After=network.target

[Service]
User=<your-username>
WorkingDirectory=/home/<your-username>/Ai-based-Data-Cleaning-and-deduplications-tool
ExecStart=/home/<your-username>/Ai-based-Data-Cleaning-and-deduplications-tool/denv/bin/streamlit run app.py
Restart=always
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable intelliclean.service
sudo systemctl start intelliclean.service
sudo systemctl status intelliclean.service

journalctl -u intelliclean.service -f
journalctl -u intelliclean.service -n 50

```

7. Accessing the Application

Once launched, the Streamlit server listens on port 8501.

Local access: <http://localhost:8501>

External Network Access: <http://<server-external-ip>:8501>

If the server is hosted on a cloud provider, ensure that firewall rules, security groups, or network ACLs allow inbound TCP traffic on port 8501.